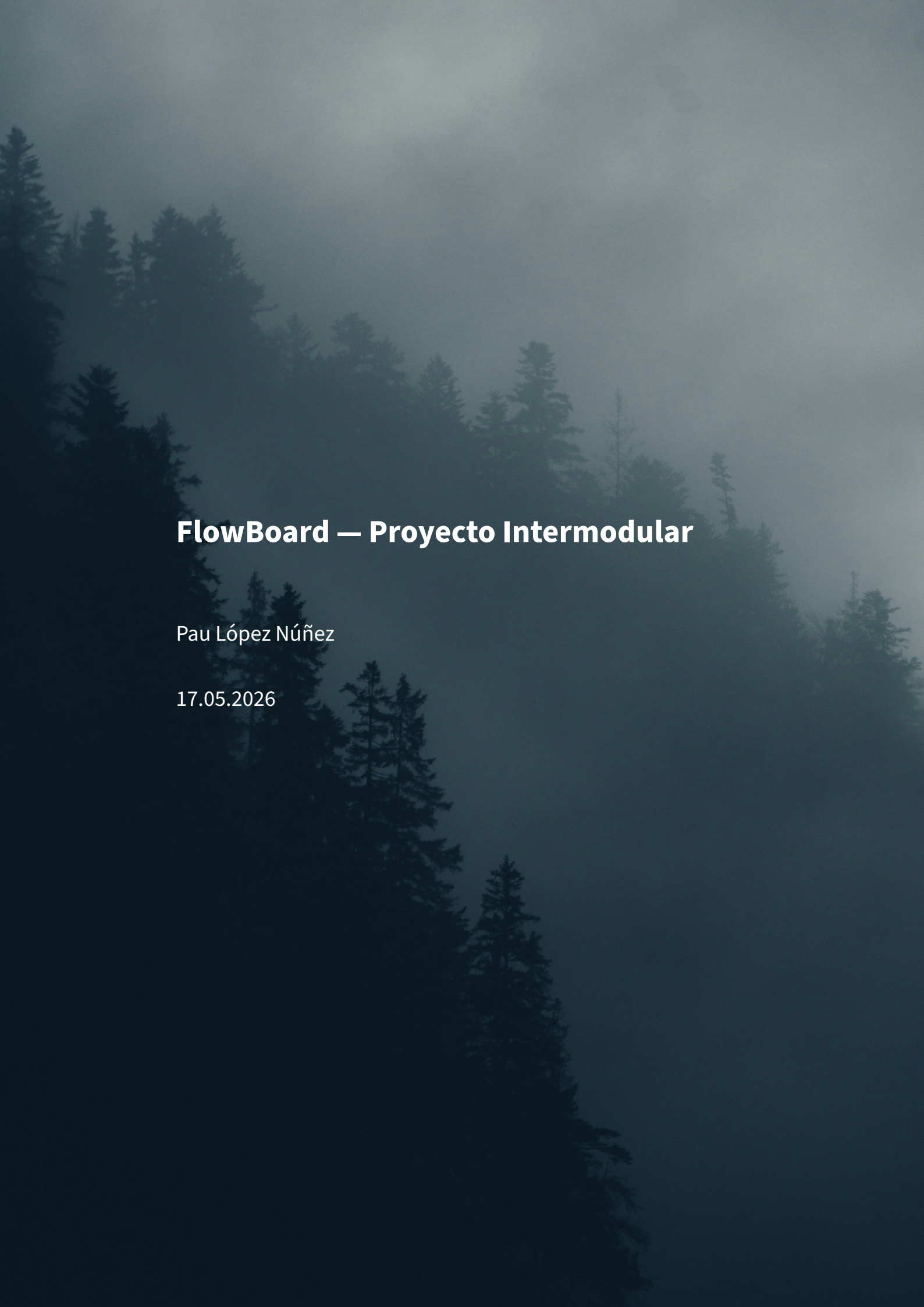




FlowBoard — Proyecto Intermodular

Pau López Núñez

17.05.2026

Índice

1. Introducción	8
1.1 Descripción general del proyecto	8
1.2 Objetivos del proyecto	9
Objetivo general	9
Objetivos específicos	9
1.3 Público objetivo	10
1.4 Alcance y limitaciones	10
Funcionalidades implementadas	10
Limitaciones	11
2. Análisis del sistema	12
2.1 Requisitos funcionales	12
RF-01: Autenticación y gestión de usuarios	12
RF-02: Gestión de tareas	12
RF-03: Gestión de proyectos	12
RF-04: Editor de documentos colaborativo	12
RF-05: Espacios de trabajo	13
RF-06: Chat integrado	13
RF-07: Notificaciones	13
RF-08: Configuración	13
RF-09: Búsqueda de texto completo	13
RF-10: Plantillas de documentos	13
RF-11: Favoritos y organización de documentos	14
RF-12: Exportación de documentos	14
RF-13: Menciones (@mentions)	14
2.2 Requisitos no funcionales	14
RNF-01: Rendimiento	14
RNF-02: Seguridad	14
RNF-03: Disponibilidad	15
RNF-04: Usabilidad	15
RNF-05: Mantenibilidad	15
2.3 Casos de uso / Historias de usuario	15
CU-01: Registrar usuario	15
CU-02: Inicio de sesión con Google	16
CU-03: Crear y editar documento colaborativo	16
CU-04: Gestionar tareas	16

CU-05: Recuperar contraseña	16
2.4 Diagrama de casos de uso	19
3. Diseño del sistema	20
3.1 Arquitectura general de la aplicación	20
3.2 Diagrama de componentes	22
Cliente Android	22
Servidor Ktor	22
3.3 Diseño de la base de datos	23
Diagrama Entidad-Relación	23
Definición de tablas principales (Exposed ORM)	23
Procedimiento almacenado — trigger set_updated_at	25
Diccionario de datos — tabla users	26
Diccionario de datos — tabla tasks	26
Diccionario de datos — tabla documents	27
Diccionario de datos — tabla projects	27
Diccionario de datos — tabla workspaces	28
Diccionario de datos — tabla messages	28
Diccionario de datos — tabla notifications	29
Diccionario de datos — tabla workspace_members	29
Diccionario de datos — tabla chat_rooms	30
Diccionario de datos — tabla chat_participants	31
Diccionario de datos — tabla board_permissions	31
Diccionario de datos — tabla password_reset_tokens	31
3.4 Diseño de la interfaz de usuario	32
Prototipo de alta fidelidad — Figma	32
Mapa de navegación	35
Sistema de temas Material 3	36
4. Tecnologías y herramientas utilizadas	38
4.1 Lenguajes de programación	38
4.2 Frameworks y librerías	38
Cliente Android	38
Servidor Ktor	39
4.3 Base de datos	40
PostgreSQL (servidor)	40
Room / SQLite (cliente Android)	40
DataStore Preferences	40

4.4 Entornos de desarrollo	40
4.5 Control de versiones y CI/CD	41
5. Implementación	42
5.1 Estructura del proyecto	42
5.2 Descripción de los módulos principales	43
Autenticación (backend)	43
Seguridad JWT (backend)	45
Pool de conexiones y esquema (backend)	45
Endpoints REST de tareas (backend)	47
Base de datos Room (cliente Android)	48
Inyección de dependencias Hilt (cliente Android)	49
ViewModel de tareas (cliente Android)	50
5.3 Funcionalidades clave implementadas	51
Motor CRDT para edición colaborativa	51
Scaffold con TopAppBar, NavigationBar e innerPadding (cliente Android)	53
Cards expandibles con animación Spring y rememberSaveable (cliente Android)	55
ContentScale.Crop en imágenes de portada de documentos	57
Modifier.weight para distribución proporcional en el Dashboard	58
rememberSaveable para preservar el estado ante rotación de pantalla	59
Exportación de documentos a PDF y Markdown	60
Carga de imágenes a Cloudinary	61
Broadcast WebSocket en tareas (backend)	61
Animaciones (cliente Android)	62
5.4 Capturas de pantalla de la aplicación	63
6. Pruebas	88
6.1 Plan de pruebas	88
6.2 Casos de prueba realizados	89
6.2.1 Pruebas unitarias del TaskViewModel (JUnit 4)	89
6.2.2 Pruebas de UI con Compose (Instrumented)	91
6.2.3 Pruebas de la API REST contra el servidor de producción	93
6.3 Resultados de las pruebas	99
6.4 Errores detectados y soluciones	99
7. Despliegue	101
7.1 Requisitos de instalación	101
Servidor (Render.com)	101
Cliente Android	101

7.2 Proceso de despliegue en servidor	101
Justificación del proveedor de nube — Render.com	101
Evidencia de funcionamiento — consultas de prueba	102
Despliegue del backend en Render.com	103
Build del APK (GitHub Actions)	103
Monitorización y observabilidad	103
7.3 Proceso de despliegue en móvil	104
Requisitos del dispositivo	104
Instalación desde GitHub Releases (método recomendado)	104
Instalación desde Android Studio (modo desarrollador)	104
Instalación mediante ADB	105
Permisos solicitados en tiempo de ejecución	105
7.4 Manual básico de usuario	105
Primeros pasos	105
Gestión de tareas	105
Editor de documentos	105
Workspaces	106
8. Planificación y gestión del proyecto	106
8.1 Metodología de desarrollo utilizada	106
8.2 Cronograma del proyecto	107
9. Conclusiones	107
9.1 Objetivos alcanzados	107
9.2 Dificultades encontradas	108
9.3 Mejoras futuras	108
9.4 Análisis de sostenibilidad	109
Tecnologías utilizadas y sus alternativas en software libre	109
Reflexión sobre el uso de grandes tecnológicas	110
Cifrado, privacidad y protección de datos	110
Plan de formación para usuarios de FlowBoard	111
9.5 Competencias profesionales y reflexión de empleabilidad	112
Contexto profesional durante el desarrollo del proyecto	112
Gestión del tiempo y planificación	112
Gestión emocional ante dificultades técnicas	112
Comunicación y trabajo en equipo simulado	113
Competencias identificadas como áreas de mejora	113

10. Anexos	113
10.1 Manual técnico	113
Requisitos del entorno de desarrollo	113
Configuración del entorno local	114
Esquema de autenticación	115
Referencia completa de la API REST	115
10.2 Repositorio y recursos	116
10.3 Referencias y recursos utilizados	117
Recursos de aprendizaje consultados	117
Repositorios de referencia	117
Estándares y especificaciones	117
10.4 Cumplimiento de Resultados de Aprendizaje (PIM)	118
11. Bibliografía	120
11.1 Referencias bibliográficas en formato APA	120
11.2 Recursos en línea (con URLs y fecha de consulta)	121
11.3 Documentación de librerías / frameworks utilizadas	121

Autor: Pau López Núñez

Tutor: Isabel Martí Romeu

Centro educativo: IES La Sénia

Curso académico: 2025–2026

Fecha de entrega: 17/05/2026

Repositorio: <https://github.com/paulopnun/flowboard>

Backend en producción: <https://flowboard-api-phrk.onrender.com>

Resumen

FlowBoard es una aplicación Android de productividad colaborativa inspirada en Notion, desarrollada íntegramente en Kotlin. Combina un editor de documentos en tiempo real basado en CRDT (*Conflict-free Replicated Data Type*) con un gestor completo de tareas, proyectos, espacios de trabajo, chat y notificaciones. La arquitectura sigue el patrón MVVM + Clean Architecture con tres capas bien diferenciadas: presentación (Jetpack Compose + Material 3), dominio (casos de uso y repositorios) y datos (Room SQLite + API REST Ktor). El backend es un servidor Ktor (Kotlin) desplegado en Render.com con PostgreSQL como base de datos relacional, autenticación JWT HMAC-256, BCrypt para hashing de contraseñas y soporte de inicio de sesión con Google mediante Credential Manager. El sistema permite la edición colaborativa simultánea de documentos a través de WebSockets con transformación operacional (OT), CI/CD automatizado con GitHub Actions, y una interfaz adaptativa con soporte completo de modo oscuro/claro.

Palabras clave: Android, Kotlin, Jetpack Compose, CRDT, WebSocket, Ktor, PostgreSQL, MVVM, Clean Architecture, Material Design 3, colaboración en tiempo real.

Abstract

FlowBoard is a collaborative productivity Android application inspired by Notion, written entirely in Kotlin. It combines a real-time document editor based on CRDT (*Conflict-free Replicated Data Type*) with a full task manager, projects, workspaces, chat, and notifications. The architecture follows the MVVM + Clean Architecture pattern with three distinct layers: presentation (Jetpack Compose + Material 3), domain (use cases and repositories) and data (Room SQLite + REST API Ktor). The backend is a Ktor (Kotlin) server deployed on Render.com with PostgreSQL as relational database, JWT HMAC-256 authentication, BCrypt for password hashing and Google Sign-In support via Credential Manager. The system enables simultaneous collaborative document editing via WebSockets with Operational Transformation (OT), automated CI/CD with GitHub Actions, and an adaptive UI with full dark/light mode support.

Keywords: Android, Kotlin, Jetpack Compose, CRDT, WebSocket, Ktor, PostgreSQL, MVVM, Clean Architecture, Material Design 3, real-time collaboration.

Índice

1. Introducción

- 1.1 Descripción general del proyecto
- 1.2 Objetivos del proyecto
- 1.3 Público objetivo
- 1.4 Alcance y limitaciones

2. Análisis del sistema

- 2.1 Requisitos funcionales
- 2.2 Requisitos no funcionales
- 2.3 Casos de uso / Historias de usuario
- 2.4 Diagrama de casos de uso

3. Diseño del sistema

- 3.1 Arquitectura general de la aplicación
- 3.2 Diagrama de componentes
- 3.3 Diseño de la base de datos
- 3.4 Diseño de la interfaz de usuario

4. Tecnologías y herramientas utilizadas

- 4.1 Lenguajes de programación
- 4.2 Frameworks y librerías
- 4.3 Base de datos
- 4.4 Entornos de desarrollo
- 4.5 Control de versiones

5. Implementación

- 5.1 Estructura del proyecto
- 5.2 Descripción de los módulos principales
- 5.3 Funcionalidades clave implementadas
- 5.4 Capturas de pantalla de la aplicación

6. Pruebas

- 6.1 Plan de pruebas

- 6.2 Casos de prueba realizados
- 6.3 Resultados de las pruebas
- 6.4 Errores detectados y soluciones

7. Despliegue

- 7.1 Requisitos de instalación
- 7.2 Proceso de despliegue en servidor
- 7.3 Proceso de despliegue en móvil
- 7.4 Manual básico de usuario

8. Planificación y gestión del proyecto

- 8.1 Metodología de desarrollo utilizada
- 8.2 Cronograma del proyecto

9. Conclusiones

- 9.1 Objetivos alcanzados
- 9.2 Dificultades encontradas
- 9.3 Mejoras futuras
- 9.4 Análisis de sostenibilidad
- 9.5 Competencias profesionales y reflexión de empleabilidad

10. Anexos

- 10.1 Manual técnico
- 10.2 Repositorio y recursos
- 10.3 Referencias y recursos utilizados
- 10.4 Cumplimiento de Resultados de Aprendizaje (PIM)

11. Bibliografía

- 11.1 Referencias bibliográficas en formato APA
- 11.2 Recursos en línea (con URLs y fecha de consulta)
- 11.3 Documentación de librerías / frameworks utilizadas

1. Introducción

1.1 Descripción general del proyecto

La idea de FlowBoard partió de una frustración bastante concreta: trabajando en GFT uso Jira para tareas, Confluence para documentación y Teams para comunicación. Son tres herramientas distintas

para lo que en la práctica es un único flujo de trabajo. Cada cambio de contexto tiene un coste real. Notion resuelve parte del problema, pero es una aplicación web-first cuya versión móvil se siente como un añadido, no como el producto principal.

FlowBoard es el intento de construir desde cero una herramienta de productividad colaborativa pensada para móvil: editor de documentos, gestor de tareas, chat y workspaces de equipo, todo en una única app nativa Android. El objetivo técnico central era conseguir que dos personas pudieran editar el mismo documento al mismo tiempo desde sus móviles sin que sus cambios se pisaran, algo que requirió implementar un motor CRDT (*Conflict-free Replicated Data Type*) con Transformación Operacional.

El sistema tiene dos partes:

- **Cliente Android:** app nativa en Kotlin 1.9 y Jetpack Compose, arquitectura MVVM + Clean Architecture con tres capas diferenciadas.
- **Servidor backend:** API REST y WebSocket con Ktor (Kotlin) desplegado en Render.com, con PostgreSQL como base de datos relacional.

1.2 Objetivos del proyecto

Objetivo general

Desarrollar una aplicación Android completa de productividad colaborativa que permita a equipos gestionar tareas, proyectos y documentos en tiempo real, aplicando los conocimientos adquiridos en todos los módulos del ciclo de Desarrollo de Aplicaciones Multiplataforma (DAM).

Objetivos específicos

1. **Acceso a Datos (AD — RA02):** Persistencia dual: PostgreSQL con Exposed ORM en el servidor (13 tablas, triggers PL/pgSQL, HikariCP) y Room SQLite en el cliente para lectura offline. Los datos de usuario viajan solo cuando hay conexión; en local se sirven desde caché.
2. **Desarrollo de Interfaces (DI — RA01/02/03/05/06):** UI declarativa con Jetpack Compose y Material Design 3. El requisito personal aquí era que la app no pareciera hecha en el ciclo: modo oscuro/claro dinámico, sidebar estilo Notion, fuente Inter y animaciones Spring en vez de las por defecto.
3. **Programación Multimedia y Móviles (PMM — RA03):** WebSockets para sincronización en tiempo real y un motor CRDT con Transformación Operacional para la edición colaborativa de documentos. El cursor animado de otros usuarios en el editor fue uno de los últimos añadidos y quizá el más visual.

4. **Programación de Procesos y Servicios (PPS — RA04/05):** Backend completamente asíncrono con coroutines de Kotlin. Más de 40 endpoints REST bajo `/api/v1/` y dos endpoints WebSockets. La autenticación JWT se valida en un interceptor antes de llegar a cualquier ruta protegida.
5. **Itinerario para la Empleabilidad II (RA02):** Google Sign-In con Credential Manager 1.3.0. Esto fue más complicado de lo esperado en dispositivos físicos porque el flujo de autorización requiere un contexto de Activity activa, algo que con Compose no es tan directo.
6. **Sostenibilidad (RA06):** Arranque lazy de la base de datos (la inicialización no bloquea el build Docker), pooling con HikariCP para reutilizar conexiones y WorkManager para tareas diferidas en el cliente.
7. **Introducción a la Nube Pública (RA04):** Despliegue en Render.com con CI/CD en GitHub Actions: cada push a master compila el APK, lo firma con el keystore de CI y crea un GitHub Release automáticamente.

1.3 Público objetivo

FlowBoard está dirigido principalmente a:

- **Estudiantes universitarios y de FP** que trabajan en proyectos en grupo y necesitan coordinar tareas y compartir documentos.
- **Equipos de desarrollo de software** que buscan una herramienta integrada de gestión de proyectos y documentación técnica.
- **Pequeños equipos de trabajo** (5–20 personas) que necesitan una solución de productividad colaborativa sin los costes de licencia de herramientas enterprise.
- **Usuarios individuales** que desean un gestor de tareas personal con capacidades de documentación avanzadas.

1.4 Alcance y limitaciones

Funcionalidades implementadas

- Autenticación completa: registro, login, Google Sign-In, recuperación de contraseña por OTP de 6 dígitos
- Gestión de tareas con prioridades (LOW/MEDIUM/HIGH/URGENT), fechas, etiquetas y asignación a usuarios
- Editor de documentos colaborativo con bloques enriquecidos: párrafo, H1–H6, listas, todo, código, cita, divisor, imagen, vídeo, audio

- **@mentions** en el editor: escribir @ despliega un DropdownMenu con usuarios activos filtrados; al seleccionar uno se inserta @username en el cursor
- **Plantillas de documentos:** 5 plantillas integradas (Meeting Notes, Project Plan, To-Do List, Weekly Review, README) accesibles desde TemplatesBottomSheet
- **Favoritos:** icono de estrella en tarjetas de documento, DocumentEntity.isStarred, flujo starredDocuments
- **Búsqueda de texto completo:** SearchScreen que busca en títulos y contenido JSON de bloques; muestra documentos recientes en estado idle
- **Duplicar documento:** crea una copia con nuevo UUID y sufijo "(copy)" desde el menú contextual
- **Exportación de documentos:** a Markdown (compartición directa) y a PDF nativo con android.graphics.pdf.PdfDocument
- **Sidebar colapsable:** panel lateral en el Dashboard con header de perfil (avatar, nombre, email, ajustes), secciones PRIVATE y SHARED WITH ME colapsables (chevron), y Papelera/Logout en área desplazable
- Espacios de trabajo (Workspaces) con invitación por código único de 12 caracteres; papelera de documentos de workspace
- Proyectos con tablero Kanban
- Chat integrado: salas directas, grupos y canales de proyecto
- Notificaciones del sistema con deep links
- Sistema de permisos (viewer/editor/owner) para documentos
- Modo oscuro/claro persistido en DataStore
- Calendario de eventos
- Asistente IA integrado
- CI/CD con GitHub Actions: build automático + release de APK en cada push a master

Limitaciones

- La aplicación está disponible únicamente para Android (API 26+, Android 8.0 Oreo o superior)
- El servidor utiliza el plan gratuito de Render.com, lo que implica arranque en frío de hasta 50 segundos tras inactividad prolongada
- La funcionalidad offline está limitada a lectura de datos cacheados en Room; la creación y edición requiere conexión
- No se implementa FCM (Firebase Cloud Messaging); las notificaciones push dependen del servidor de notificaciones de Android

2. Análisis del sistema

2.1 Requisitos funcionales

RF-01: Autenticación y gestión de usuarios

- **RF-01.1:** El sistema permitirá el registro con email, nombre de usuario y contraseña.
- **RF-01.2:** El sistema permitirá el inicio de sesión con credenciales email/contraseña.
- **RF-01.3:** El sistema ofrecerá inicio de sesión con Google mediante Google Credential Manager.
- **RF-01.4:** El sistema permitirá la recuperación de contraseña mediante OTP de 6 dígitos enviado por email, con validez de 15 minutos.
- **RF-01.5:** Las contraseñas se cifrarán con BCrypt antes de almacenarse.
- **RF-01.6:** El sistema emitirá un token JWT HMAC-256 en cada inicio de sesión exitoso.

RF-02: Gestión de tareas

- **RF-02.1:** Los usuarios podrán crear tareas con título, descripción, prioridad, fecha de vencimiento, etiquetas y asignación.
- **RF-02.2:** Las tareas podrán marcarse como completadas o pendientes.
- **RF-02.3:** Las tareas podrán asociarse a proyectos.
- **RF-02.4:** Las tareas podrán definirse como eventos de calendario con hora de inicio, hora de fin y ubicación.
- **RF-02.5:** Los cambios se propagarán en tiempo real a todos los usuarios del mismo proyecto mediante WebSocket.

RF-03: Gestión de proyectos

- **RF-03.1:** Los usuarios podrán crear proyectos con nombre, descripción, color y fecha límite.
- **RF-03.2:** Los proyectos podrán tener múltiples miembros.
- **RF-03.3:** Cada proyecto dispondrá de un tablero Kanban.

RF-04: Editor de documentos colaborativo

- **RF-04.1:** Los usuarios podrán crear y editar documentos con bloques de contenido enriquecido.
- **RF-04.2:** Los cambios se sincronizarán en tiempo real mediante WebSocket y el motor CRDT.

- **RF-04.3:** El sistema resolverá conflictos de edición concurrente mediante Transformación Operacional (OT).
- **RF-04.4:** Se mostrará la posición del cursor de cada editor activo.
- **RF-04.5:** Los documentos podrán ser privados, compartidos o asociados a un workspace.

RF-05: Espacios de trabajo

- **RF-05.1:** Los usuarios podrán crear workspaces e invitar a otros mediante código único de 12 caracteres.
- **RF-05.2:** Los workspaces tendrán roles: OWNER, ADMIN y MEMBER.

RF-06: Chat integrado

- **RF-06.1:** El sistema dispondrá de salas de chat directas (1:1) y grupales.
- **RF-06.2:** Los mensajes podrán responderse en hilo y contener adjuntos.

RF-07: Notificaciones

- **RF-07.1:** El sistema generará notificaciones para asignaciones, menciones e invitaciones.
- **RF-07.2:** Las notificaciones incluirán deep links al recurso relacionado.

RF-08: Configuración

- **RF-08.1:** El usuario podrá activar/desactivar el modo oscuro, persistido en DataStore.
- **RF-08.2:** El usuario podrá editar su perfil y cambiar su contraseña.

RF-09: Búsqueda de texto completo

- **RF-09.1:** La aplicación dispondrá de una pantalla de búsqueda que filtre documentos por título y contenido de bloques.
- **RF-09.2:** En estado inactivo se mostrarán los documentos accedidos recientemente.
- **RF-09.3:** Los resultados indicarán el número de coincidencias encontradas.

RF-10: Plantillas de documentos

- **RF-10.1:** El sistema ofrecerá una galería de 5 plantillas integradas: Meeting Notes, Project Plan, To-Do List, Weekly Review y README.

- **RF-10.2:** Al seleccionar una plantilla, el título y los bloques iniciales se precargarán en el editor.

RF-11: Favoritos y organización de documentos

- **RF-11.1:** Los usuarios podrán marcar documentos como favoritos mediante un icono de estrella.
- **RF-11.2:** Los documentos favoritos aparecerán destacados en el listado.
- **RF-11.3:** Los usuarios podrán duplicar un documento, generando una copia con sufijo "(copy)".

RF-12: Exportación de documentos

- **RF-12.1:** Los usuarios podrán exportar cualquier documento a formato **Markdown** (.md) y compartirlo mediante el intent del sistema.
- **RF-12.2:** Los usuarios podrán exportar cualquier documento a **PDF** nativo mediante `android.graphics.pdf.PdfDocument`, con opción de guardar en ubicación elegida por el usuario (Storage Access Framework).

RF-13: Menciones (@mentions)

- **RF-13.1:** Al escribir @ en cualquier bloque del editor colaborativo, el sistema mostrará un menú desplegable con los usuarios activos del workspace filtrado por texto.
- **RF-13.2:** Al seleccionar un usuario, se insertará @username en la posición del cursor.

2.2 Requisitos no funcionales

RNF-01: Rendimiento

- Carga inicial de lista de tareas < 2 segundos en condiciones normales de red.
- El motor CRDT aplicará operaciones locales de forma síncrona (< 16 ms) para garantizar 60 FPS.
- HikariCP mantendrá hasta 10 conexiones activas al servidor PostgreSQL.

RNF-02: Seguridad

- Todas las comunicaciones usarán HTTPS (TLS 1.2+).
- Las contraseñas se almacenarán con BCrypt (factor de coste predeterminado de la librería jBCrypt).

- El servidor implementará protección anti-enumeración en el endpoint de recuperación de contraseña.
- Las rutas protegidas devolverán HTTP 401 ante token inválido o ausente.

RNF-03: Disponibilidad

- Backend desplegado en Render.com con disponibilidad del 99 % (plan gratuito con posible cold start).
- La aplicación funcionará en modo lectura offline gracias a la caché Room.

RNF-04: Usabilidad

- La interfaz seguirá las guías de Material Design 3.
- Soporte completo de modo oscuro y claro con transición dinámica.
- Todas las transiciones de navegación tendrán duración de 280 ms.
- Compatible con Android API 26+.

RNF-05: Mantenibilidad

- Patrón MVVM + Clean Architecture con separación estricta de capas.
- Inyección de dependencias con Hilt.
- API REST versionada en `/api/v1/`.

2.3 Casos de uso / Historias de usuario

CU-01: Registrar usuario

- **Actor:** Usuario no autenticado
- **Precondición:** El usuario no tiene cuenta
- **Flujo principal:**
 1. El usuario introduce email, nombre de usuario, nombre completo y contraseña
 2. El sistema valida que no existan duplicados de email o username
 3. El sistema cifra la contraseña con BCrypt
 4. El sistema almacena el usuario y devuelve un token JWT
 5. El usuario accede al Dashboard
- **Flujo alternativo A:** Email o username ya existen → error 400 Bad Request (ver nota TC-02)

CU-02: Inicio de sesión con Google

- **Actor:** Usuario no autenticado
- **Flujo principal:**
 1. El usuario pulsa “Continuar con Google”
 2. Credential Manager muestra el selector de cuentas
 3. El sistema recibe el token de Google y hace upsert del usuario
 4. El sistema devuelve un token JWT propio

CU-03: Crear y editar documento colaborativo

- **Actor:** Usuario autenticado con rol editor o superior
- **Flujo principal:**
 1. El usuario crea un nuevo documento
 2. El sistema abre el editor y establece conexión WebSocket
 3. El usuario añade bloques de contenido
 4. El motor CRDT genera operaciones y las envía al servidor
 5. El servidor retransmite las operaciones a todos los colaboradores
- **Flujo alternativo A:** Dos usuarios editan el mismo bloque → OT resuelve el conflicto de forma determinista

CU-04: Gestionar tareas

- **Actor:** Usuario autenticado
- **Flujo principal:**
 1. El usuario accede a la lista de tareas
 2. Crea una tarea con título, descripción, prioridad y fecha opcional
 3. La tarea se persiste en la API REST y se replica en la caché Room
 4. Si pertenece a un proyecto, el servidor emite evento WebSocket TASK_CREATED

CU-05: Recuperar contraseña

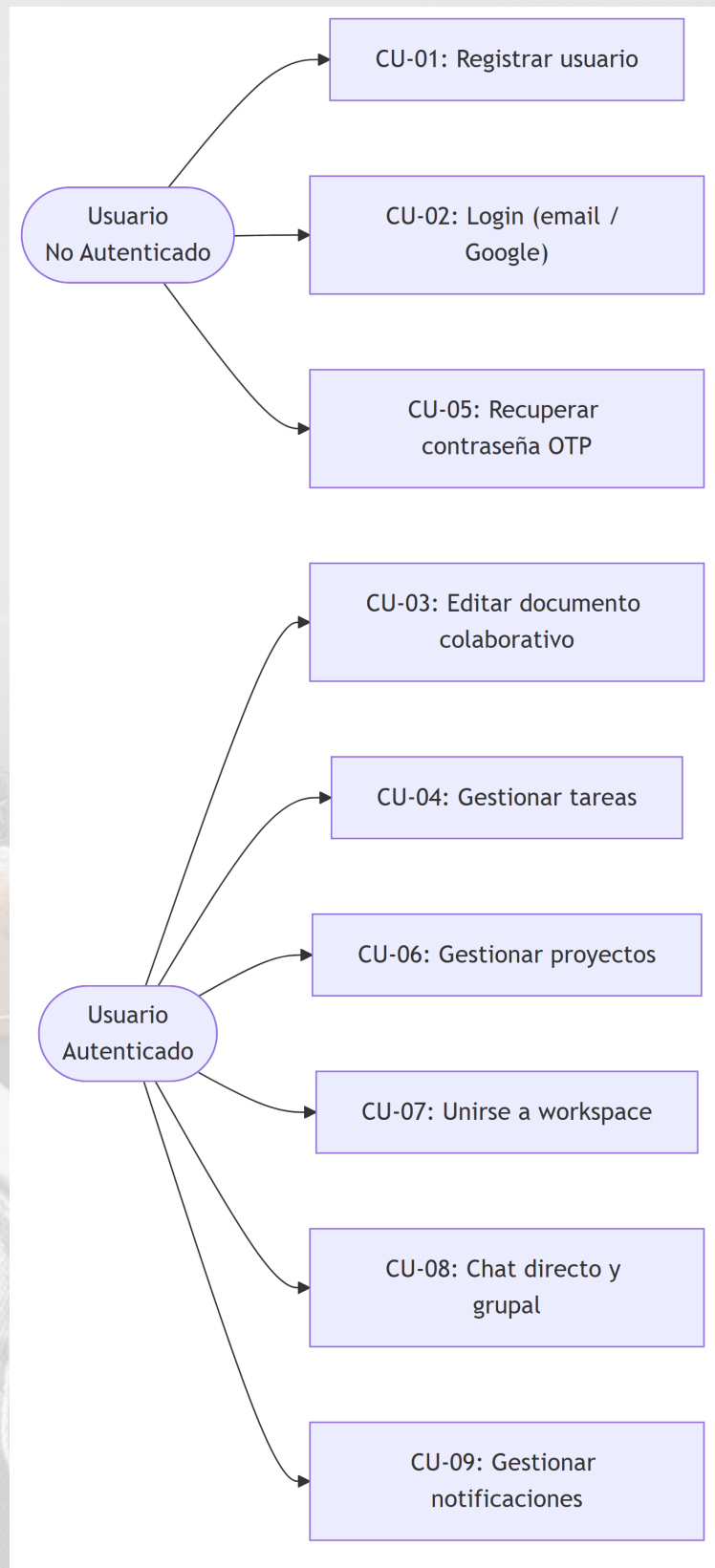
- **Actor:** Usuario no autenticado
- **Flujo principal:**
 1. El usuario introduce su email

2. El sistema genera un OTP de 6 dígitos con validez de 15 minutos
3. El sistema envía el OTP por email (Resend API)
4. El usuario introduce el OTP y la nueva contraseña
5. El sistema valida el OTP y actualiza el hash BCrypt





2.4 Diagrama de casos de uso



3. Diseño del sistema

3.1 Arquitectura general de la aplicación

FlowBoard implementa una arquitectura cliente-servidor. El cliente Android sigue el patrón **MVVM + Clean Architecture** en tres capas y el servidor Ktor una arquitectura **por capas** (rutas → dominio → datos).

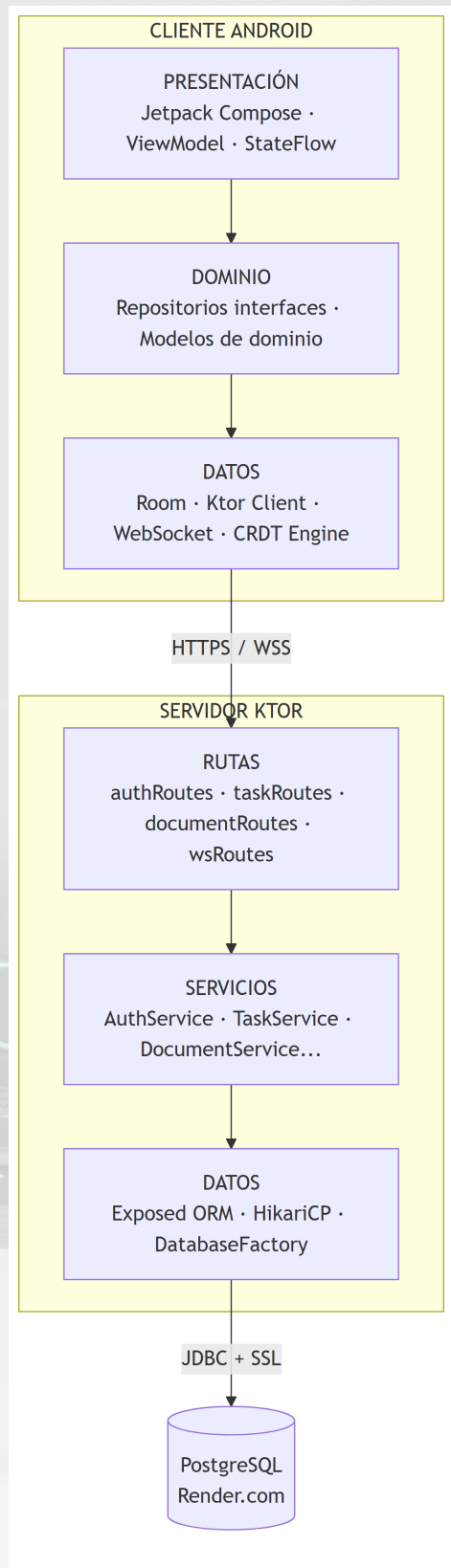


Diagrama de arquitectura — MVVM + Clean Architecture + Ktor

3.2 Diagrama de componentes

Cliente Android

Componente	Tecnología	Responsabilidad
FlowBoardApp.kt	NavHost Compose	Host de navegación con 20+ rutas
TaskViewModel	ViewModel + Hilt	Estado y lógica de tareas
CollaborativeDocumentViewModel	ViewModel + Hilt	Estado del editor colaborativo
CRDTEngine	Kotlin Singleton	Motor de Transformación Operacional
FlowBoardDatabase	Room	Base de datos local SQLite
NetworkModule	Hilt Module	Inyección de HttpClient + WebSocket
TaskWebSocketClient	Ktor WS Client	Canal WebSocket para tareas
FlowBoardTheme	Compose Theme	Tema Material 3 dinámico

Servidor Ktor

Componente	Tecnología	Responsabilidad
Application.kt	Ktor	Entrada + cadena de plugins
AuthService.kt	Kotlin object	Registro, login, Google, OTP
TaskService.kt	Kotlin class	CRUD tareas + broadcast WS
WebSocketManager	Kotlin class	Gestión de salas WebSocket
DatabaseFactory.kt	HikariCP	Pool de conexiones PostgreSQL
Tables.kt	Exposed ORM	Definición de 13 tablas
JwtConfig	Auth0 JWT	Firma y verificación HMAC-256

3.3 Diseño de la base de datos

Diagrama Entidad-Relación

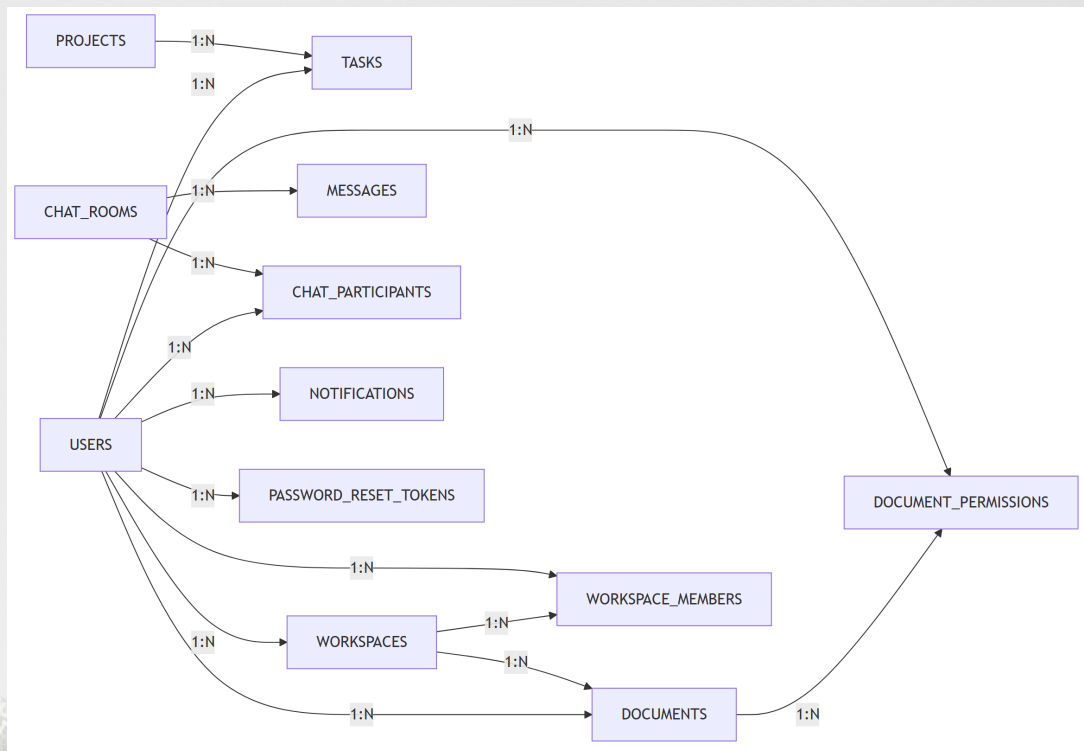


Diagrama Entidad-Relación — FlowBoard

Definición de tablas principales (Exposed ORM)

```
// Users table
object Users : UUIDTable("users") {
    val email = varchar("email", 255).uniqueIndex()
    val username = varchar("username", 100).uniqueIndex()
    val fullName = varchar("full_name", 255)
    val passwordHash = varchar("password_hash", 255)
    val role = enumeration("role",
        ↳ UserRole::class).default(UserRole.USER)
    val profileImageUrl = varchar("profile_image_url", 500).nullable()
    val isActive = bool("is_active").default(true)
    val createdAt = datetime("created_at")
    val lastLoginAt = datetime("last_login_at").nullable()
}
```


// Tasks table – also supports calendar events

```
object Tasks : UUIDTable("tasks") {  
    val title          = varchar("title", 255)  
    val description    = text("description")  
    val isCompleted    = bool("is_completed").default(false)  
    val priority       = enumeration("priority",  
        ↪ TaskPriority::class).default(TaskPriority.MEDIUM)  
    val dueDate        = datetime("due_date").nullable()  
    val createdAt      = datetime("created_at")  
    val updatedAt      = datetime("updated_at")  
    val assignedTo     = uuid("assigned_to").nullable()  
    val projectId      = uuid("project_id").nullable()  
    val tags           = json<List<String>>("tags",  
        ↪ Json.Default).default(emptyList())  
    val attachments    = json<List<String>>("attachments",  
        ↪ Json.Default).default(emptyList())  
    val isEvent        = bool("is_event").default(false)  
    val eventStartTime = datetime("event_start_time").nullable()  
    val eventEndTime  = datetime("event_end_time").nullable()  
    val location       = varchar("location", 500).nullable()  
    val createdBy      = uuid("created_by")  
}
```

// Documents table – supports parent-child hierarchy

```
object Documents : UUIDTable("documents") {  
    val title          = varchar("title", 500)  
    val content        = text("content")  
    val ownerId        = uuid("owner_id")  
    val parentId       = uuid("parent_id").nullable()  
    val isPublic       = bool("is_public").default(false)  
    val visibility     = varchar("visibility", 20).default("private")  
    val workspaceId    = uuid("workspace_id").nullable()  
    val createdAt      = datetime("created_at")  
    val updatedAt      = datetime("updated_at")  
    val lastEditedBy   = uuid("last_edited_by").nullable()  
}
```

// Document permissions – viewer / editor / owner roles

```
object DocumentPermissions : UUIDTable("document_permissions") {  
    val documentId = uuid("document_id")  
    val userId      = uuid("user_id")  
    val role        = varchar("role", 50).default("viewer")  
    val grantedBy   = uuid("granted_by")
```



```
    val grantedAt = datetime("granted_at")
}

// Workspaces – unique 12-character invite code
object Workspaces : UUIDTable("workspaces") {
    val name = varchar("name", 255)
    val description = text("description").nullable()
    val ownerId = uuid("owner_id")
    val inviteCode = varchar("invite_code", 12).uniqueIndex()
    val createdAt = datetime("created_at")
    val updatedAt = datetime("updated_at")
}

// Password reset OTP – expires in 15 min
object PasswordResetTokens : UUIDTable("password_reset_tokens") {
    val email = varchar("email", 255).index()
    val code = varchar("code", 6)
    val expiresAt = datetime("expires_at")
    val used = bool("used").default(false)
}
```

Procedimiento almacenado — trigger set_updated_at

Para garantizar que el campo `updated_at` de las tablas `tasks` y `documents` se actualiza automáticamente en cada modificación sin depender del código de aplicación, se define una función PL/pgSQL y un trigger en PostgreSQL:

```
-- Stored function: updates the updated_at column to current timestamp
CREATE OR REPLACE FUNCTION set_updated_at()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = NOW();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Trigger on tasks table
CREATE TRIGGER tasks_set_updated_at
    BEFORE UPDATE ON tasks
    FOR EACH ROW
    EXECUTE FUNCTION set_updated_at();

-- Trigger on documents table
```



```
CREATE TRIGGER documents_set_updated_at
BEFORE UPDATE ON documents
FOR EACH ROW
EXECUTE FUNCTION set_updated_at();
```

Este procedimiento se ejecuta a nivel de base de datos, por lo que Exposed ORM no necesita gestionar el campo `updated_at` manualmente en cada `update { }`. La función `set_updated_at()` es el único stored procedure del esquema; el resto de la lógica de negocio reside en los servicios Ktor para mantener la separación de responsabilidades.

Diccionario de datos — tabla `users`

Campo	Tipo	Nulo	Descripción
<code>id</code>	UUID (PK)	No	Identificador único autogenerado
<code>email</code>	VARCHAR(255)	No	Email único del usuario
<code>username</code>	VARCHAR(100)	No	Nombre de usuario único
<code>full_name</code>	VARCHAR(255)	No	Nombre completo
<code>password_hash</code>	VARCHAR(255)	No	Hash BCrypt de la contraseña
<code>role</code>	ENUM	No	USER / ADMIN
<code>profile_image_url</code>	VARCHAR(500)	Sí	URL de la foto de perfil
<code>is_active</code>	BOOLEAN	No	Estado de la cuenta
<code>created_at</code>	DATETIME	No	Fecha de registro
<code>last_login_at</code>	DATETIME	Sí	Último inicio de sesión

Diccionario de datos — tabla `tasks`

Campo	Tipo	Nulo	Descripción
<code>id</code>	UUID (PK)	No	Identificador único
<code>title</code>	VARCHAR(255)	No	Título de la tarea
<code>description</code>	TEXT	No	Descripción detallada
<code>is_completed</code>	BOOLEAN	No	Estado completado

Campo	Tipo	Nulo	Descripción
priority	ENUM	No	LOW / MEDIUM / HIGH / URGENT
due_date	DATETIME	Sí	Fecha de vencimiento
assigned_to	UUID (FK→users)	Sí	Usuario asignado
project_id	UUID (FK→projects)	Sí	Proyecto al que pertenece
tags	JSON	No	Array de etiquetas
is_event	BOOLEAN	No	Si es evento de calendario
event_start_time	DATETIME	Sí	Inicio del evento
event_end_time	DATETIME	Sí	Fin del evento
location	VARCHAR(500)	Sí	Ubicación del evento
created_by	UUID (FK→users)	No	Usuario creador

Diccionario de datos — tabla documents

Campo	Tipo	Nulo	Descripción
id	UUID (PK)	No	Identificador único
title	VARCHAR(500)	No	Título del documento
content	TEXT	No	Contenido serializado (JSON de bloques)
owner_id	UUID (FK→users)	No	Propietario
parent_id	UUID (FK→documents)	Sí	Documento padre
visibility	VARCHAR(20)	No	private / shared / workspace
workspace_id	UUID (FK→workspaces)	Sí	Workspace al que pertenece
last_edited_by	UUID (FK→users)	Sí	Último editor

Diccionario de datos — tabla projects

Campo	Tipo	Nulo	Descripción
id	UUID (PK)	No	Identificador único
name	VARCHAR(255)	No	Nombre del proyecto
description	TEXT	Sí	Descripción detallada
color	VARCHAR(7)	Sí	Color hexadecimal del proyecto
due_date	DATETIME	Sí	Fecha límite del proyecto
owner_id	UUID (FK→users)	No	Usuario creador/propietario
created_at	DATETIME	No	Fecha de creación

Diccionario de datos — tabla workspaces

Campo	Tipo	Nulo	Descripción
id	UUID (PK)	No	Identificador único
name	VARCHAR(255)	No	Nombre del workspace
description	TEXT	Sí	Descripción del workspace
owner_id	UUID (FK→users)	No	Usuario propietario
invite_code	VARCHAR(12)	No	Código único de invitación (índice único)
created_at	DATETIME	No	Fecha de creación
updated_at	DATETIME	No	Fecha de última modificación

Diccionario de datos — tabla messages

Campo	Tipo	Nulo	Descripción
id	UUID (PK)	No	Identificador único

Campo	Tipo	Nulo	Descripción
content	TEXT	No	Contenido del mensaje
sender_id	UUID (FK→users)	No	Usuario que envía el mensaje
room_id	UUID (FK→chat_rooms)	No	Sala de chat a la que pertenece
reply_to	UUID (FK→messages)	Sí	Mensaje al que responde (hilo)
created_at	DATETIME	No	Fecha de envío

Diccionario de datos — tabla notifications

Campo	Tipo	Nulo	Descripción
id	UUID (PK)	No	Identificador único
user_id	UUID (FK→users)	No	Usuario destinatario
type	VARCHAR(50)	No	Tipo: TASK_ASSIGNED / MENTION / INVITE
title	VARCHAR(255)	No	Título de la notificación
body	TEXT	No	Cuerpo del mensaje
deep_link	VARCHAR(500)	Sí	Ruta de navegación destino
is_read	BOOLEAN	No	Estado de lectura
created_at	DATETIME	No	Fecha de generación

Diccionario de datos — tabla workspace_members

Campo	Tipo	Nulo	Descripción
id	UUID (PK)	No	Identificador único
workspace_id	UUID (FK→workspaces)	No	Workspace al que pertenece

Campo	Tipo	Nulo	Descripción
user_id	UUID (FK→users)	No	Usuario miembro
role	VARCHAR(50)	No	Rol: OWNER / ADMIN / MEMBER
joined_at	DATETIME	No	Fecha de incorporación al workspace

Diccionario de datos — tabla chat_rooms

Campo	Tipo	Nulo	Descripción
id	UUID (PK)	No	Identificador único
type	VARCHAR(50)	No	Tipo: DIRECT / GROUP / PROJECT / TASK_THREAD
name	VARCHAR(255)	Sí	Nombre de la sala (nulo en chats directos)
description	TEXT	Sí	Descripción opcional
resource_id	UUID	Sí	ID del recurso asociado (proyecto, tarea...)
resource_type	VARCHAR(50)	Sí	Tipo del recurso asociado
created_by	UUID (FK→users)	No	Usuario que crea la sala
created_at	DATETIME	No	Fecha de creación
updated_at	DATETIME	No	Fecha del último mensaje
is_archived	BOOLEAN	No	Indica si la sala está archivada

Diccionario de datos — tabla chat_participants

Campo	Tipo	Nulo	Descripción
id	UUID (PK)	No	Identificador único
chat_room_id	UUID (FK→chat_rooms)	No	Sala a la que pertenece
user_id	UUID (FK→users)	No	Usuario participante
role	VARCHAR(50)	No	Rol en la sala: OWNER / ADMIN / MEMBER
joined_at	DATETIME	No	Fecha de incorporación
last_read_at	DATETIME	Sí	Último mensaje leído por el usuario
is_muted	BOOLEAN	No	Silencia las notificaciones de la sala

Diccionario de datos — tabla board_permissions

Campo	Tipo	Nulo	Descripción
id	UUID (PK)	No	Identificador único
board_id	UUID (FK→projects)	No	Tablero al que aplica el permiso
user_id	UUID (FK→users)	No	Usuario al que se concede el permiso

Diccionario de datos — tabla password_reset_tokens

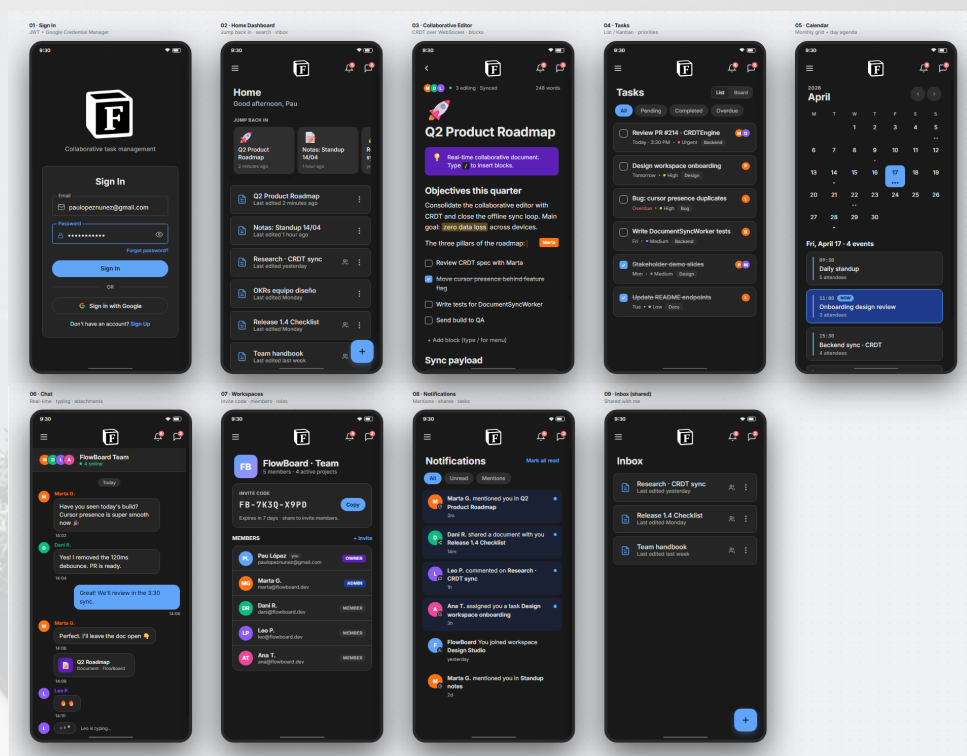
Campo	Tipo	Nulo	Descripción
id	UUID (PK)	No	Identificador único
email	VARCHAR(255)	No	Email del usuario que solicita el reset
code	VARCHAR(6)	No	OTP de 6 dígitos enviado por correo
expires_at	DATETIME	No	Expiración del token (15 minutos)
used	BOOLEAN	No	Indica si el token ya fue consumido

3.4 Diseño de la interfaz de usuario

Prototipo de alta fidelidad — Figma

El diseño de FlowBoard partió de un prototipado previo en Figma donde se definieron los flujos de usuario y la distribución visual de cada pantalla antes de la implementación. Se diseñaron dos variantes del tema: modo oscuro (predeterminado) y modo claro, asegurando que la paleta semántica y los componentes Material 3 fueran coherentes en ambos.

Figura 3.1 — Wireframes modo oscuro (Figma)



El prototipo en modo oscuro muestra 9 pantallas principales con fondo #191919 y texto blanco:

Pantalla

Elementos clave del prototipo

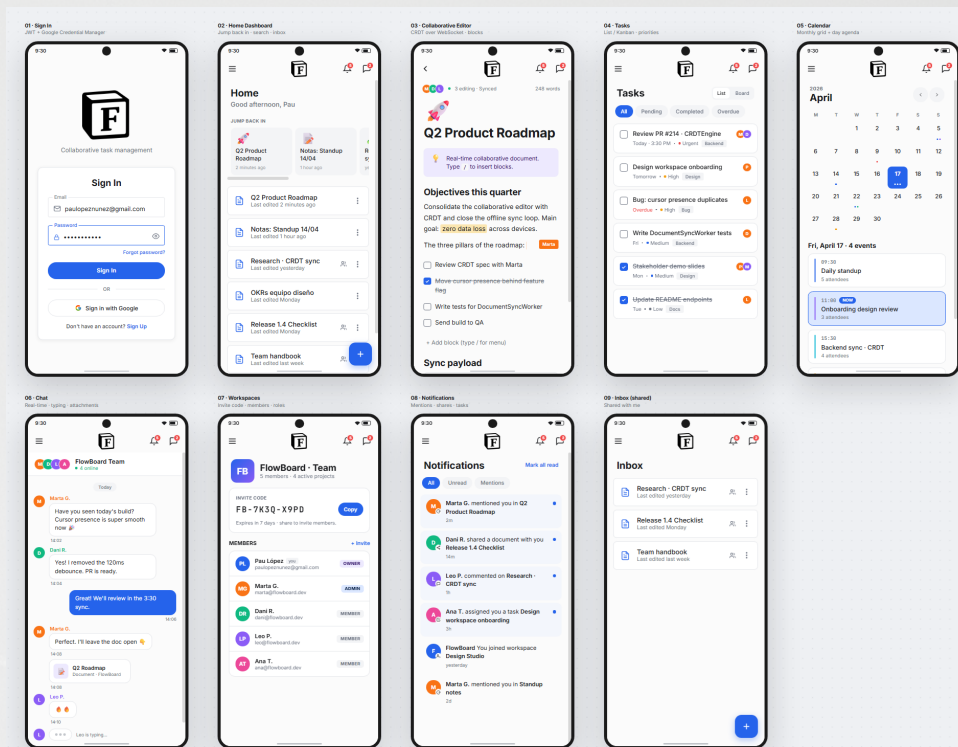
01 Sign In

Cubo “F” centrado, campos Email/Password, botón “Sign In” azul, “Sign in with Google” outline



Pantalla	Elementos clave del prototipo
02 Home Dashboard	Saludo contextual, carrusel “JUMP BACK IN”, lista de documentos recientes con menú de tres puntos
03 Collaborative Editor	Cabecera “Editing · Screen-Sharing” en verde, toolbar de bloques, cursores multi-usuario, contenido rich text
04 Tasks	Vistas Pending/Completed/Overdue con tarjetas Kanban, avatares de asignados, indicadores de prioridad
05 Calendar	Cuadrícula mensual, eventos del día seleccionado en sidebar lateral, integración con tareas
06 Chat	Burbujas diferenciadas (propio/ajeno), avatares, timestamps, campo de entrada con emoji y adjuntos
07 Workspaces	Workspace “FlowBoard - Team” con código de invitación FB-7K32-E9P0, lista de miembros con roles
08 Notifications	Feed de actividad: menciones, documentos compartidos, asignaciones de tarea, uniones a workspace
09 Inbox	Centro de menciones personales con hilo de conversación y estado leído/no leído

Figura 3.2 — Wireframes modo claro (Figma)



El mismo conjunto de 9 pantallas en tema claro: fondos #FFFFFF/#F5F5F5, texto oscuro #1A1A1A, manteniendo idéntica distribución de componentes. La coherencia entre temas se garantiza mediante el sistema de tokens de color de Material 3 (colorScheme.surface, colorScheme.onSurface, etc.) que mapean automáticamente según el modo activo en DataStore.

Decisiones de diseño tomadas en la fase de prototipado:

- **Barra lateral colapsable** (en lugar de NavigationBar inferior) para maximizar el área de contenido, inspirada en Notion y Linear.
- **Cards de acceso rápido fijas** (“JUMP BACK IN”) de 150×100 dp con TextOverflow.Ellipsis, para que el grid no se rompa con títulos largos.
- **Editor de bloques con toolbar contextual** T / H1 / H2 / H3 / B / I / U en la parte superior, sin ocultar el teclado.
- **Paleta semántica de prioridades:** Low (azul) / Medium (gris) / High (púrpura) / Urgent (rojo con icono de alerta).
- **Indicador “● Live”** en verde en la TopAppBar del editor cuando el WebSocket está conectado.

Pantallas principales diseñadas:

Pantalla	Descripción del diseño
Login / Registro	Campos centrados, fondo oscuro, botón Google outline
Dashboard	Carrusel “JUMP BACK IN” + lista documentos recientes
Editor de documentos	Toolbar de bloques + indicador Live + FAB IA
Gestión de tareas	TabRow All/Pending/Completed + FAB creación
Calendario	Cuadrícula mensual + lista de eventos del día
Chat	Burbujas diferenciadas + timestamps
Workspaces	Lista con código de invitación + menú contextual
Notificaciones	Feed con chips de filtro
Perfil / Ajustes	Dark mode toggle + gestión de cuenta

Mapa de navegación

```
app_start
+-- login
+-- register
+-- forgot_password
+-- (autenticado)
  +-- dashboard
  +-- task_list
  +-- task_detail/{taskId}
  +-- create_task
  +-- calendar
  +-- project_list
  +-- project_detail/{projectId}
  +-- my_documents
  +-- document_editor/{documentId}
  +-- collaborative_document/{documentId}
  +-- search
  +-- workspace_list
```



```
+-- workspace_detail/{workspaceId}
+-- workspace_documents/{workspaceId}
+-- chat
+-- chat_room/{chatRoomId}
+-- notifications
+-- profile
`-- settings
```

Sistema de temas Material 3

Paleta de colores (Color.kt):

```
// Dark theme – Notion-style background
private val DarkColorScheme = darkColorScheme(
    primary          = Color(0xFF3B82F6),
    onPrimary        = Color(0xFFFFFFFF),
    primaryContainer  = Color(0xFF1D3A6E),
    background       = Color(0xFF191919),
    surface          = Color(0xFF1F1F1F),
    onBackground     = Color(0xFFE8E8E8),
    onSurface        = Color(0xFFE8E8E8)
)

// Light theme
private val LightColorScheme = lightColorScheme(
    primary          = Color(0xFF2563EB),
    onPrimary        = Color(0xFFFFFFFF),
    background       = Color(0xFFFFFFFF),
    surface          = Color(0xFFFFF8F9FA),
    onBackground     = Color(0xFF191919),
    onSurface        = Color(0xFF191919)
)

// Semantic priority colours (updated palette – matches Create Task screen)
val PriorityLow     = Color(0xFF4CAF50)    // green
val PriorityMedium  = Color(0xFF2196F3)    // blue
val PriorityHigh    = Color(0xFF7C3AED)    // purple (updated from orange)
val PriorityUrgent  = Color(0xFFFF4336)    // red

// Cursor colours for real-time collaboration
val CollabCursorColors = listOf(
    Color(0xFFEF4444), Color(0xFF3B82F6), Color(0xFF10B981),
```



```
        Color(0xFF59E0B), Color(0xFF8B5CF6), Color(0xFFEC4899)
    )
```

Tipografía Inter (Type.kt):

```
// Full M3 type scale using Inter (Google Fonts)
```

```
val Typography = Typography(
    titleLarge = TextStyle(
        fontFamily = InterFontFamily,
        fontWeight = FontWeight.SemiBold,
        fontSize = 22.sp,
        lineHeight = 28.sp
    ),
    bodyLarge = TextStyle(
        fontFamily = InterFontFamily,
        fontWeight = FontWeight.Normal,
        fontSize = 16.sp,
        lineHeight = 24.sp
    ),
    labelSmall = TextStyle(
        fontFamily = InterFontFamily,
        fontWeight = FontWeight.Medium,
        fontSize = 11.sp,
        lineHeight = 16.sp,
        letterSpacing = 0.5.sp
    )
    // 13 styles total: displayLarge → labelSmall
)
```

Radios de curvatura (Shapes.kt):

```
// Shapes.kt – corner radius scale aligned with Material 3 tokens
```

```
val Shapes = Shapes(
    extraSmall = RoundedCornerShape(4.dp),
    small = RoundedCornerShape(8.dp),
    medium = RoundedCornerShape(12.dp),
    large = RoundedCornerShape(16.dp),
    extraLarge = RoundedCornerShape(28.dp)
)
```

Tema dinámico (Theme.kt):

```
@Composable
fun FlowBoardTheme(
    settingsViewModel: SettingsViewModel = hiltViewModel(),
```



```
        content: @Composable () -> Unit
    ) {
        val darkModePreference by
            settingsViewModel.darkModeEnabled.collectAsState()
        val darkTheme = darkModePreference ?: isSystemInDarkTheme()
        val colorScheme = if (darkTheme) DarkColorScheme else LightColorScheme

        MaterialTheme(
            colorScheme = colorScheme,
            typography = Typography,
            shapes      = Shapes,    // corner radii applied globally
            content     = content
        )
    }
}
```

4. Tecnologías y herramientas utilizadas

4.1 Lenguajes de programación

Lenguaje	Versión	Uso
Kotlin	1.9.x	Cliente Android + servidor Ktor
SQL	PostgreSQL 15	Base de datos relacional
YAML	—	CI/CD GitHub Actions

4.2 Frameworks y librerías

Cliente Android

Librería	Versión	Propósito
Jetpack Compose	BOM 2024.x	UI declarativa
Material 3	1.2.x	Sistema de diseño
Hilt	2.50	Inyección de dependencias
Room	2.6.x	Base de datos local SQLite

Librería	Versión	Propósito
Ktor Client	2.3.7	HTTP + WebSocket
DataStore Preferences	1.0.x	Persistencia de preferencias
Google Credential Manager	1.3.0	Autenticación con Google (actualizado para soporte en dispositivo físico)
Navigation Compose	2.7.x	Navegación entre pantallas
Kotlinx Coroutines	1.7.x	Concurrencia asíncrona
Kotlinx Serialization	1.6.x	Serialización JSON
Kotlinx DateTime	0.5.x	Manejo de fechas
Coil Compose	2.5.x	Carga de imágenes asíncrona
WorkManager	2.9.x	Tareas en segundo plano
android.graphics.pdf.PdfDocument\$SDK		Generación de PDFs nativos para exportación de documentos
Cloudinary REST API	—	Almacenamiento y entrega de imágenes en la nube (HTTP directo, sin SDK)

Servidor Ktor

Librería	Versión	Propósito
Ktor Server (Netty)	2.3.7	Framework web + motor
Exposed ORM	0.46.x	ORM para PostgreSQL
HikariCP	5.x	Pool de conexiones
PostgreSQL Driver	42.x	Driver JDBC
jBCrypt	0.4	Hash de contraseñas
Auth0 JWT	4.x	Generación y verificación JWT
kotlinx-datetime	0.5.x	Fechas en servidor

Librería	Versión	Propósito
Logback	1.4.x	Sistema de logging

4.3 Base de datos

PostgreSQL (servidor)

- **Proveedor:** Render.com (PostgreSQL 15, plan gratuito)
- **ORM:** JetBrains Exposed con DSL de tabla
- **Pool:** HikariCP, máx. 10 conexiones, TRANSACTION_REPEATABLE_READ
- **13 tablas:** Users, Tasks, Projects, BoardPermissions, Documents, DocumentPermissions, Notifications, PasswordResetTokens, Workspaces, WorkspaceMembers, ChatRooms, ChatParticipants, Messages

Room / SQLite (cliente Android)

- **ORM:** Room 2.6.x con DAOs tipados
- **11 entidades:** TaskEntity, UserEntity, ProjectEntity, NotificationEntity, ChatRoomEntity, MessageEntity, ChatParticipantEntity, TypingIndicatorEntity, DocumentEntity, PendingOperationEntity, WorkspaceEntity
- **Versión del esquema:** 9

DataStore Preferences

- **Uso:** Token JWT, userId, boardId, preferencia de modo oscuro
- **Implementación:** Preferences DataStore con claves tipadas

4.4 Entornos de desarrollo

Herramienta	Uso
Android Studio Iguana	IDE principal Android
IntelliJ IDEA	Desarrollo backend Ktor
Gradle 8.x	Sistema de build

Herramienta	Uso
Git / GitHub	Control de versiones + CI/CD
Render.com	Hosting backend + PostgreSQL
Postman	Pruebas de API REST

4.5 Control de versiones y CI/CD

Pipeline de GitHub Actions (`.github/workflows/build-apk.yml`):

```
name: Build APK

on:
  push:
    branches: [ master ]
  pull_request:
    branches: [ master ]
  workflow_dispatch:

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Set up JDK 17
        uses: actions/setup-java@v4
        with:
          java-version: '17'
          distribution: 'temurin'

      - name: Cache Gradle
        uses: actions/cache@v4
        with:
          path: |
            ~/.gradle/caches
            ~/.gradle/wrapper
          key: gradle-${{ hashFiles('**/*.gradle*') }}

      - name: Build debug APK
        working-directory: android
```



```
run: ./gradlew assembleDebug --no-daemon

- name: Upload APK artifact
  uses: actions/upload-artifact@v4
  with:
    name: FlowBoard-debug-${{ github.sha }}
    path: android/app/build/outputs/apk/debug/app-debug.apk
    retention-days: 30

- name: Create GitHub Release
  uses: softprops/action-gh-release@v2
  if: github.ref == 'refs/heads/master'
  with:
    tag_name: build-${{ github.run_number }}
    name: FlowBoard v1.0 (build ${{ github.run_number }})
    files: android/app/build/outputs/apk/debug/app-debug.apk
```

Cada push a master compila el APK y crea automáticamente un GitHub Release con el APK adjunto.

5. Implementación

5.1 Estructura del proyecto

Módulo	Capa	Contenido principal
android/	data/	auth, crdt (CRDTEngine), local (Room: dao, entities), remote (api, dto, websocket), repository, workers
android/	di/	Módulos Hilt: Database, Network, Repository, CRDT
android/	domain/	model/ (User, Task, Document...), repository/ (interfaces)

Módulo	Capa	Contenido principal
android/	presentation/	ui/screens/ (auth, tasks, documents, projects, profile, settings), ui/util/ (CloudinaryUploader, DateUtils), viewmodel/ (15+ ViewModels)
android/	theme/	Color.kt, Theme.kt, Type.kt (Material 3)
android/	raíz	FlowBoardApp.kt, FlowBoardApplication.kt, MainActivity.kt
backend/	data/	database/ (Tables.kt, DatabaseFactory.kt), models/
backend/	domain/	AuthService, TaskService, DocumentService...
backend/	plugins/	Security, Routing, Database, WebSockets (Ktor config)
backend/	routes/	Endpoints REST por recurso
backend/	raíz	Application.kt
infra/		.github/workflows/build-apk.yml, docs/

5.2 Descripción de los módulos principales

Autenticación (backend)

```
// AuthService.kt – Registration with BCrypt hashing and JWT issuance
suspend fun register(request: RegisterRequest): LoginResponse = dbQuery {
    val existing = Users
        .select { Users.email eq request.email or (Users.username eq
            ↪ request.username) }
        .firstOrNull()
    if (existing != null) throw IllegalArgumentException("User already
        ↪ exists")
}
```



```
val hashedPassword = BCrypt.hashpw(request.password, BCrypt.gensalt())
val userId = UUID.randomUUID()

Users.insert {
    it[id] = userId
    it[email] = request.email
    it[username] = request.username
    it[passwordHash] = hashedPassword
    it[createdAt] =
↪ Clock.System.now().toLocalDateTime(TimeZone.currentSystemDefault())
}

LoginResponse(
    token = JwtConfig.makeToken(request.email, userId.toString(),
↪ request.username),
    user = User(id = userId.toString(), email = request.email, ...)
)
}

// Password recovery – anti-enumeration: always returns true regardless of
↪ email existence
suspend fun requestPasswordReset(email: String): Boolean = dbQuery {
    val expires = Clock.System.now().plus(15,
↪ DateTimeUnit.MINUTE).toLocalDateTime(TimeZone.UTC)
    val userExists = Users.select { Users.email eq email }.count() > 0

    if (userExists) {
        PasswordResetTokens.update({ PasswordResetTokens.email eq email }) {
            it[used] = true // Invalida tokens anteriores
        }
        val code = (100000..999999).random().toString()
        PasswordResetTokens.insert {
            it[PasswordResetTokens.email] = email
            it[PasswordResetTokens.code] = code
            it[PasswordResetTokens.expiresAt] = expires
        }
        GlobalScope.launch { EmailService.sendPasswordResetEmail(email,
↪ code) }
    }
    true // Siempre devuelve true (anti-enumeración)
}
```


Seguridad JWT (backend)

```
// Security.kt – JWT plugin with HMAC-256 verification
fun Application.configureSecurity() {
    install(Authentication) {
        jwt("auth-jwt") {
            realm = JwtConfig.realm
            verifier(JwtConfig.verifier)
            validate { credential ->
                val userId = credential.payload.getClaim("userId")?.asString()
                if (!userId.isNullOrEmpty())
                    ⇨ JWTPrincipal(credential.payload) else null
            }
        }
    }
}

object JwtConfig {
    private val secret = System.getenv("JWT_SECRET") ?: "dev-secret-key"
    private val algorithm = Algorithm.HMAC256(secret)

    val verifier = JWT.require(algorithm)
        .withIssuer(System.getenv("JWT_ISSUER") ?: "flowboard-api")
        .withAudience(System.getenv("JWT_AUDIENCE") ?: "flowboard-app")
        .build()

    fun makeToken(email: String, userId: String, username: String = ""):
        ⇨ String =
        JWT.create()
            .withClaim("email", email)
            .withClaim("userId", userId)
            .withClaim("username", username)
            .sign(algorithm)
}
```

Pool de conexiones y esquema (backend)

```
// DatabaseFactory.kt – HikariCP + Render.com URL parsing
object DatabaseFactory {
    fun init() {
        val database = Database.connect(createHikariDataSource())
        transaction(database) {
            SchemaUtils.create(

```



```
        Users, Tasks, Projects, BoardPermissions,
        Documents, DocumentPermissions, Notifications,
        PasswordResetTokens, Workspaces, WorkspaceMembers,
        ChatRooms, ChatParticipants, Messages
    )
}
}

private fun createHikariDataSource(): HikariDataSource {
    val databaseUrl = System.getenv("DATABASE_URL")
    val config = HikariConfig().apply {
        driverClassName = "org.postgresql.Driver"

        if (databaseUrl?.startsWith("postgresql://") == true) {
            // Render.com delivers: postgresql://user:pass@host/db
            // JDBC requires:      jdbc:postgresql://host/db
            val regex = Regex("postgresql://(?:[a-zA-Z_0-9-]+):(?:[a-zA-Z_0-9-]+)@(?:[a-zA-Z_0-9-]+)/(?:[a-zA-Z_0-9-]+)")
            val (user, pass, host, db) =
                ⇨ regex.find(databaseUrl)!!.destructured
            val externalHost = if (host.startsWith("dpg-"))
                "$host.oregon-postgres.render.com:5432" else "$host:5432"
            this.jdbcUrl =
                ⇨ "jdbc:postgresql://$externalHost/$db?sslmode=require"
            this.username = user
            this.password = pass
        } else {
            this.jdbcUrl = databaseUrl ?:
                ⇨ "jdbc:postgresql://localhost:5432/flowboard"
            this.username = System.getenv("DATABASE_USER") ?: "flowboard"
            this.password = System.getenv("DATABASE_PASSWORD") ?:
                ⇨ "flowboard"
        }

        maximumPoolSize = 10
        isAutoCommit = false
        transactionIsolation = "TRANSACTION_REPEATABLE_READ"
        connectionTimeout = 30_000
        idleTimeout = 600_000
        maxLifetime = 1_800_000
    }
    return HikariDataSource(config)
}
```



```
suspend fun <T> dbQuery(block: suspend () -> T): T =  
    newSuspendedTransaction(Dispatchers.IO) { block() }  
}
```

Endpoints REST de tareas (backend)

// TaskRoutes.kt – full JWT-protected CRUD

```
fun Route.taskRoutes(taskService: TaskService) {  
    authenticate("auth-jwt") {  
        route("/tasks") {  
  
            get { // GET /api/v1/tasks  
                val userId = call.principal<JWTPrincipal>()  
                    ?.payload?.getClaim("userId")?.asString()  
                ?: return@get call.respond(HttpStatusCode.Unauthorized)  
                call.respond(HttpStatusCode.OK,  
↪ taskService.getAllTasksForUser(userId))  
            }  
  
            post { // POST /api/v1/tasks  
                val userId = call.principal<JWTPrincipal>()  
                    ?.payload?.getClaim("userId")?.asString()!!  
                val request = call.receive<CreateTaskRequest>()  
                val task = taskService.createTask(request, userId)  
                call.respond(HttpStatusCode.Created, task)  
            }  
  
            put("/{id}") { // PUT /api/v1/tasks/{id}  
                val taskId = call.parameters["id"]!!  
                val userId = call.principal<JWTPrincipal>()  
                    ?.payload?.getClaim("userId")?.asString()!!  
                val request = call.receive<UpdateTaskRequest>()  
                val task = taskService.updateTask(taskId, request, userId)  
                if (task != null) call.respond(HttpStatusCode.OK, task)  
                else call.respond(HttpStatusCode.NotFound)  
            }  
  
            delete("/{id}") { // DELETE /api/v1/tasks/{id}  
                val taskId = call.parameters["id"]!!  
                val userId = call.principal<JWTPrincipal>()  
                    ?.payload?.getClaim("userId")?.asString()!!  
                val deleted = taskService.deleteTask(taskId, userId)  
                if (deleted) call.respond(HttpStatusCode.NoContent)  
            }  
        }  
    }  
}
```



```

        else call.respond(HttpStatusCode.NotFound)
    }

    patch("/{id}/toggle") { // PATCH /api/v1/tasks/{id}/toggle
        val taskId = call.parameters["id"]!!
        val userId = call.principal<JWTPrincipal>()
            ?.payload?.getClaim("userId")?.asString()!!
        val task = taskService.toggleTaskStatus(taskId, userId)
        if (task != null) call.respond(HttpStatusCode.OK, task)
        else call.respond(HttpStatusCode.NotFound)
    }
}
}
}
}

```

Base de datos Room (cliente Android)

```

// FlowBoardDatabase.kt - 11 entities, schema version 9
@Database(
    entities = [
        TaskEntity::class, UserEntity::class, ProjectEntity::class,
        NotificationEntity::class, ChatRoomEntity::class,
        ↪ MessageEntity::class,
        ChatParticipantEntity::class, TypingIndicatorEntity::class,
        DocumentEntity::class, PendingOperationEntity::class,
        ↪ WorkspaceEntity::class
    ],
    version = 9,
    exportSchema = false
)
@TypeConverters(Converters::class)
abstract class FlowBoardDatabase : RoomDatabase() {
    abstract fun taskDao(): TaskDao
    abstract fun documentDao(): DocumentDao
    abstract fun pendingOperationDao(): PendingOperationDao
    abstract fun workspaceDao(): WorkspaceDao

    companion object {
        @Volatile private var INSTANCE: FlowBoardDatabase? = null

        fun getDatabase(context: Context): FlowBoardDatabase =
            INSTANCE ?: synchronized(this) {

```



```
        Room.databaseBuilder(context, FlowBoardDatabase::class.java,
        ↪ "flowboard_database")
            .fallbackToDestructiveMigration()
            .build()
            .also { INSTANCE = it }
    }
}
}
```

Inyección de dependencias Hilt (cliente Android)

```
// NetworkModule.kt – Hilt @Singleton providers for HTTP and WebSocket
↪ clients
```

```
@Module
```

```
@InstallIn(SingletonComponent::class)
```

```
object NetworkModule {
```

```
    @Provides @Singleton
```

```
    fun provideHttpClient(): HttpClient = HttpClient(Android) {
        engine { connectTimeout = 30_000; socketTimeout = 30_000 }
        install(ContentNegotiation) {
            json(Json { isLenient = true; ignoreUnknownKeys = true })
        }
        install(Logging) { level = LogLevel.HEADERS }
        expectSuccess = false
    }
}
```

```
// WebSocket client with OkHttp engine – supports Android native
↪ WebSocket
```

```
@Provides @Singleton @WebSocketClientQualifier
```

```
fun provideWebSocketClient(): HttpClient = HttpClient(OkHttp) {
    install(WebSockets) {
        pingInterval = 30_000 // keep-alive ping every 30 s
        maxFrameSize = Long.MAX_VALUE
    }
    install(Logging) { level = LogLevel.INFO }
}
}
```

```
@Provides @Singleton
```

```
fun provideTaskApiService(
    @HttpClientQualifier httpClient: HttpClient,
    authRepository: AuthRepository
): TaskApiService = TaskApiService(httpClient, authRepository)
```



```
}
```

ViewModel de tareas (cliente Android)

```
// TaskViewModel.kt — MVVM with StateFlow and coroutines
@HiltViewModel
class TaskViewModel @Inject constructor(
    private val taskRepository: TaskRepositoryImpl,
    private val authRepository: AuthRepository
) : ViewModel() {

    private val _uiState = MutableStateFlow(TaskUiState())
    val uiState: StateFlow<TaskUiState> = _uiState.asStateFlow()

    // Room flows — automatically re-emit when the database changes
    val allTasks = taskRepository.getAllTasks()
        .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000),
            ↪ emptyList())

    val pendingTasks = taskRepository.getTasksByStatus(false)
        .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000),
            ↪ emptyList())

    fun createTask(title: String, description: String, priority:
        ↪ TaskPriority = TaskPriority.MEDIUM, ...) {
        viewModelScope.launch {
            _uiState.update { it.copy(isLoading = true) }
            taskRepository.createTask(task).fold(
                onSuccess = { _uiState.update { it.copy(isLoading = false,
                ↪ message = "Task created successfully") } },
                onFailure = { e -> _uiState.update { it.copy(isLoading =
                ↪ false, error = e.message) } }
            )
        }
    }

    fun toggleTaskStatus(taskId: String) {
        viewModelScope.launch {
            taskRepository.toggleTaskStatus(taskId).fold(
                onSuccess = { _uiState.update { it.copy(message = "Task
                ↪ status updated") } },
                onFailure = { e -> _uiState.update { it.copy(error =
                ↪ e.message) } }
            )
        }
    }
}
```



```

    )
  }
}

fun connectToBoard(boardId: String, token: String, userId: String) {
    viewModelScope.launch { taskRepository.connectToBoard(boardId,
    ↪ token, userId) }
}

fun disconnectFromBoard() {
    viewModelScope.launch { taskRepository.disconnectFromBoard() }
}

data class TaskUiState(
    val isLoading: Boolean = false,
    val selectedTask: Task? = null,
    val error: String? = null,
    val message: String? = null
)

```

5.3 Funcionalidades clave implementadas

Motor CRDT para edición colaborativa

```

// CRDTEngine.kt — Operational Transformation for conflict-free
    ↪ collaborative editing
@Singleton
class CRDTEngine @Inject constructor() {

    private val _document = MutableStateFlow<CollaborativeDocument?>(null)
    val document: StateFlow<CollaborativeDocument?> =
        ↪ _document.asStateFlow()

    private val vectorClock = mutableMapOf<String, Long>()
    private val operationHistory = mutableListOf<DocumentOperation>()
    private val appliedOperations = mutableSetOf<String>() // idempotency
        ↪ guard

    fun applyOperation(operation: DocumentOperation): Boolean {
        if (appliedOperations.contains(operation.operationId)) return false

        val currentDoc = _document.value ?: return false
    }
}

```



```

val newBlocks = when (operation) {
  is AddBlockOperation      ->
    ↪ handleAddBlock(currentDoc.blocks, operation)
  is DeleteBlockOperation   ->
    ↪ handleDeleteBlock(currentDoc.blocks, operation)
  is UpdateBlockContentOperation ->
    ↪ handleUpdateContent(currentDoc.blocks, operation)
  is UpdateBlockFormattingOperation ->
    ↪ handleUpdateFormatting(currentDoc.blocks, operation)
  is MoveBlockOperation     ->
    ↪ handleMoveBlock(currentDoc.blocks, operation)
  is CursorMoveOperation    -> currentDoc.blocks // cursor
    ↪ moves do not mutate the document
  else                     -> currentDoc.blocks
}

_document.value = currentDoc.copy(blocks = newBlocks)
appliedOperations.add(operation.operationId)
operationHistory.add(operation)
return true
}

// OT transform – adjusts position of concurrent insert operations
private fun transformContentOperations(
  op1: UpdateBlockContentOperation,
  op2: UpdateBlockContentOperation
): UpdateBlockContentOperation {
  if (op1.blockId != op2.blockId) return op1

  val newPosition = when {
    op2.position < op1.position ->
      op1.position + op2.content.length // op2 inserted before op1
    ↪ → shift right
    op2.position == op1.position ->
      if (op1.operationId < op2.operationId) op1.position
      else op1.position + op2.content.length // deterministic
        ↪ tie-break by operation ID
    else -> op1.position
  }
  return op1.copy(position = newPosition)
}

```



```

fun mergeRemoteOperations(remoteOps: List<DocumentOperation>):
    ↪ List<DocumentOperation> {
        val localOps = getPendingOperations()
        return remoteOps
            .filter { !appliedOperations.contains(it.operationId) }
            .map { remoteOp ->
                transformOperation(remoteOp, localOps).also {
                    ↪ applyOperation(it) }
            }
    }

fun reset() {
    _document.value = null
    vectorClock.clear()
    operationHistory.clear()
    appliedOperations.clear()
}

```

Scaffold con TopAppBar, NavigationBar e innerPadding (cliente Android)

Nota de evolución: el prototipo inicial usaba NavigationBar inferior con 4 destinos. En la implementación final se migró a un ModalNavigationDrawer lateral (sidebar colapsable al estilo Notion), visible en las capturas §5.4. El fragmento a continuación muestra la estructura de Scaffold con TopAppBar, NavigationBar e innerPadding, que sigue siendo el patrón base de Compose aunque en la versión final el bottomBar fue sustituido por el drawer.

```

// MainScaffold.kt – Scaffold structure with M3 TopAppBar, NavigationBar and
    ↪ innerPadding
@Composable
fun MainScaffold(navController: NavHostController) {
    val currentRoute = navContro-
        ↪ ller.currentBackStackEntryAsState().value?.destination?.route

    Scaffold(
        topBar = {
            TopAppBar(
                title = { Text("FlowBoard", style =
                    ↪ MaterialTheme.typography.titleLarge) },
                colors = TopAppBarDefaults.topAppBarColors(
                    containerColor = MaterialTheme.colorScheme.surface,
                    titleContentColor = MaterialTheme.colorScheme.onSurface
                )
            )
        }
    )
}

```



```

        ),
        actions = {
            IconButton(onClick = {
                ↪ navController.navigate("notifications") }) {
                Icon(Icons.Default.Notifications, contentDescription
                ↪ = "Notifications")
            }
        }
    ),
    },
    bottomBar = {
        NavigationBar(containerColor =
        ↪ MaterialTheme.colorScheme.surface) {
            listOf(
                Triple("dashboard", Icons.Default.Home,
                ↪ "Home"),
                Triple("task_list", Icons.Default.CheckBox,
                ↪ "Tasks"),
                Triple("my_documents", Icons.Default.Description,
                ↪ "Docs"),
                Triple("workspace_list", Icons.Default.Group,
                ↪ "Spaces")
            ).forEach { (route, icon, label) ->
                NavigationBarItem(
                    icon = { Icon(icon, contentDescription = label) },
                    label = { Text(label) },
                    selected = currentRoute == route,
                    onClick = { navController.navigate(route) }
                )
            }
        }
    ) { innerPadding ->
        // innerPadding prevents content from being hidden behind TopAppBar
        ↪ / NavigationBar
        NavHost(
            navController = navController,
            startDestination = "dashboard",
            modifier = Modifier.padding(innerPadding)
        )
    }
}

```


Cards expandibles con animación Spring y rememberSaveable (cliente Android)

```
// TaskCard.kt – Spring-physics expansion + rememberSaveable for rotation
↳ resilience
@Composable
fun TaskCard(task: Task, onClick: () -> Unit, modifier: Modifier = Modifier)
↳ {
    // rememberSaveable preserves expanded state across device rotation
    var expanded by rememberSaveable { mutableStateOf(false) }

    val cardElevation by animateDpAsState(
        targetValue = if (expanded) 8.dp else 2.dp,
        animationSpec = spring(
            dampingRatio = Spring.DampingRatioMediumBouncy,
            stiffness = Spring.StiffnessLow
        ),
        label = "card_elevation"
    )

    Card(
        modifier = modifier
            .fillMaxWidth()
            .clickable { expanded = !expanded },
        elevation = CardDefaults.cardElevation(defaultElevation =
↳ cardElevation),
        shape = MaterialTheme.shapes.medium
    ) {
        Column {
            Row(
                modifier = Modifier.fillMaxWidth().padding(16.dp),
                verticalAlignment = Alignment.CenterVertically
            ) {
                // Modifier.weight(1f) – title column takes all available
                ↳ horizontal space
                Column(modifier = Modifier.weight(1f)) {
                    Text(task.title, style =
↳ MaterialTheme.typography.titleMedium)
                    Text(
                        text = task.priority.name,
                        style = MaterialTheme.typography.labelSmall,
                        color = when (task.priority) {
                            TaskPriority.LOW -> PriorityLow
                            TaskPriority.MEDIUM -> PriorityMedium
                        }
                    )
                }
            }
        }
    }
}
```



```

        TaskPriority.HIGH    -> PriorityHigh
        TaskPriority.URGENT -> PriorityUrgent
    }
)
}
Icon(
    imageVector      = if (expanded) Icons.Default.ExpandLess
                      else           Icons.Default.ExpandMore,
    contentDescription = if (expanded) "Collapse" else
↳ "Expand"
)
}

// Spring-based enter/exit for natural physical feel
AnimatedVisibility(
    visible = expanded,
    enter   = expandVertically(
        animationSpec = spring(
            dampingRatio = Spring.DampingRatioMediumBouncy,
            stiffness     = Spring.StiffnessMedium
        )
    ) + fadeIn(spring(stiffness = Spring.StiffnessMedium)),
    exit    = shrinkVertically(spring(stiffness =
↳ Spring.StiffnessMedium)) + fadeOut()
    ) {
        Column(modifier = Modifier.padding(horizontal = 16.dp,
↳ vertical = 8.dp)) {
            if (task.description.isNotBlank()) {
                Text(task.description, style =
↳ MaterialTheme.typography.bodyMedium)
                Spacer(modifier = Modifier.height(8.dp))
            }
            task.dueDate?.let {
                Text(
                    text = "Due:
↳ ${it.format(DateTimeFormatter.ofPattern("dd MMM yyyy"))}",
                    style = MaterialTheme.typography.labelMedium,
                    color = MaterialTheme.colorScheme.onSurfaceVariant
                )
            }
        }
    }
}
}
}

```



```
}  
}
```

ContentScale.Crop en imágenes de portada de documentos

```
// DocumentCoverImage.kt – ContentScale.Crop fills the frame without  
↳ distortion  
@Composable  
fun DocumentCoverImage(imageUrl: String?, modifier: Modifier = Modifier) {  
    Box(  
        modifier = modifier  
            .fillMaxWidth()  
            .height(180.dp)  
            .clip(MaterialTheme.shapes.medium)  
    ) {  
        if (imageUrl != null) {  
            AsyncImage(  
                model = ImageRequest.Builder(LocalContext.current)  
                    .data(imageUrl)  
                    .crossfade(true)  
                    .build(),  
                contentDescription = null,  
                contentScale = ContentScale.Crop, // fills bounds,  
↳ crops excess  
                modifier = Modifier.fillMaxSize()  
            )  
        } else {  
            Box(  
                modifier = Modifier  
                    .fillMaxSize()  
                    .background(MaterialTheme.colorScheme.primaryContainer),  
                contentAlignment = Alignment.Center  
            ) {  
                Icon(  
                    imageVector = Icons.Default.Description,  
                    contentDescription = null,  
                    tint =  
↳ MaterialTheme.colorScheme.onPrimaryContainer,  
                    modifier = Modifier.size(48.dp)  
                )  
            }  
        }  
    }  
}
```



```
}
```

Modifier.weight para distribución proporcional en el Dashboard

```
// DashboardScreen.kt – Modifier.weight for adaptive proportional layout
@Composable
fun DashboardScreen(viewModel: DashboardViewModel = hiltViewModel()) {
    val uiState by viewModel.uiState.collectAsState()

    Column(modifier = Modifier.fillMaxSize().padding(16.dp)) {

        // Stats row – 25% of vertical space, two cards split 50/50 with
        // ↳ weight(1f)
        Row(
            modifier = Modifier.fillMaxWidth().weight(0.25f),
            horizontalArrangement = Arrangement.spacedBy(12.dp)
        ) {
            StatCard(
                title = "Pending tasks",
                value = uiState.pendingCount.toString(),
                modifier = Modifier.weight(1f)
            )
            StatCard(
                title = "Active projects",
                value = uiState.projectCount.toString(),
                modifier = Modifier.weight(1f)
            )
        }

        Spacer(modifier = Modifier.height(12.dp))

        // Recent tasks – remaining 75% of vertical space
        Text("Recent tasks", style = MaterialTheme.typography.titleMedium)
        Spacer(modifier = Modifier.height(8.dp))

        LazyColumn(modifier = Modifier.weight(0.75f)) {
            items(uiState.recentTasks, key = { it.id }) { task ->
                TaskCard(task = task, onClick = {}, modifier =
                // ↳ Modifier.animateItem())
            }
        }
    }
}
```


rememberSaveable para preservar el estado ante rotación de pantalla

```
// TaskListScreen.kt – rememberSaveable keeps tab and search query across
↳ config changes
@Composable
fun TaskListScreen(viewModel: TaskViewModel = hiltViewModel(), onNavigate:
↳ (String) -> Unit) {
    // All three survive device rotation and process death
    var selectedTab by rememberSaveable { mutableIntStateOf(0) }
    var searchQuery by rememberSaveable { mutableStateOf("") }
    var showCompleted by rememberSaveable { mutableStateOf(true) }

    val allTasks by viewModel.allTasks.collectAsState()
    val pendingTasks by viewModel.pendingTasks.collectAsState()

    val displayedTasks = when (selectedTab) {
        1 -> pendingTasks
        2 -> allTasks.filter { it.isCompleted }
        3 -> allTasks.filter { it.dueDate != null &&
            it.dueDate.isBefore(LocalDateTime.now()) &&
↳ !it.isCompleted }
        else -> allTasks
    }.filter { it.title.contains(searchQuery, ignoreCase = true) }

    Scaffold(
        floatingActionButton = {
            FloatingActionButton(onClick = { onNavigate("create_task") }) {
                Icon(Icons.Default.Add, contentDescription = "New task")
            }
        }
    ) { innerPadding ->
        Column(modifier = Modifier.padding(innerPadding)) {
            OutlinedTextField(
                value = searchQuery,
                onValueChange = { searchQuery = it },
                modifier = Modifier.fillMaxWidth().padding(16.dp),
                placeholder = { Text("Search tasks...") },
                leadingIcon = { Icon(Icons.Default.Search, null) },
                singleLine = true
            )
            ScrollableTabRow(selectedTabIndex = selectedTab) {
                listOf("All", "Pending", "Done", "Overdue").forEachIndexed {
↳ i, label ->
```



```

        Tab(selected = selectedTab == i, onClick = { selectedTab
        ↪ = i },
            text = { Text(label) })
        }
    }
    LazyColumn(modifier = Modifier.fillMaxSize()) {
        items(displayedTasks, key = { it.id }) { task ->
            TaskCard(task = task, onClick = {
        ↪ onNavigate("task_detail/${task.id}") },
                modifier = Modifier.animateItem())
        }
    }
}

```

Exportación de documentos a PDF y Markdown

```

// DocumentExporter.kt – native PDF generation with
↪ android.graphics.pdf.PdfDocument
fun exportToPdf(blocks: List<ContentBlock>, title: String, context: Context)
↪ {
    val file = File(context.cacheDir, suggestedPdfFileName(title))
    FileOutputStream(file).use { writePdfToStream(blocks, it) }
    shareFile(file, "application/pdf", "Export as PDF", context)
}

// savePdfToUri writes to a user-picked location (Storage Access Framework)
fun savePdfToUri(blocks: List<ContentBlock>, context: Context, uri: Uri):
↪ Boolean {
    val outputStream = context.contentResolver.openOutputStream(uri) ?:
    ↪ return false
    return runCatching { outputStream.use { writePdfToStream(blocks, it) } }
    ↪ }.isSuccess
}

fun exportToMarkdown(blocks: List<ContentBlock>, title: String, context:
↪ Context) {
    val sb = StringBuilder("# $title\n\n")
    blocks.forEach { block ->
        val line = when (block.type) {
            "h1" -> "# ${block.content}"
            "h2" -> "## ${block.content}"

```



```

        "h3" -> "### ${block.content}"
        "bullet" -> "- ${block.content}"
        "todo" -> "- [ ] ${block.content}"
        "code" -> "```\n${block.content}\n```"
        "quote" -> "> ${block.content}"
        else -> block.content
    }
    sb.appendLine(line)
}
// shares the resulting .md file via Android Intent
shareFile(createTempTextFile(sb.toString(), title, context),
    ↪ "text/markdown", "Export as Markdown", context)
}

```

Carga de imágenes a Cloudinary

```

// CloudinaryUploader.kt – uploads compressed images to Cloudinary via HTTP
↪ REST (no SDK)
object CloudinaryUploader {
    private const val UPLOAD_URL =
        ↪ "https://api.cloudinary.com/v1_1/$CLOUD_NAME/image/upload"

    suspend fun uploadImage(context: Context, uri: Uri, maxSide: Int = 512,
        ↪ quality: Int = 82): String? =
        withContext(Dispatchers.IO) {
            val dataUrl = imageUriToCompressedDataUrl(context, uri, maxSide,
                ↪ quality)
            ↪ ?: return@withContext null
            uploadDataUrl(dataUrl) // returns the HTTPS URL of the uploaded
        ↪ image
        }
}

```

Broadcast WebSocket en tareas (backend)

```

// TaskService.kt – TASK_CREATED event broadcast to all board members via
↪ WebSocket
suspend fun createTask(request: CreateTaskRequest, userId: String): Task {
    val task = dbQuery { /* inserción Exposed */ }

    if (websocketManager != null && task.projectId != null) {
        websocketManager.broadcastToRoom(

```



```

        boardId = task.projectId!!,
        message = TaskCreatedMessage(
            timestamp =
↪ Clock.System.now().toLocalDateTime(TimeZone.currentSystemDefault()),
            boardId   = task.projectId!!,
            task      = task.toSnapshot()
        )
    }
    return task
}

```

Animaciones (cliente Android)

```

// FlowBoardApp.kt – Uniform 280 ms transition across all routes
composable(
    route = "task_list",
    enterTransition = { slideInHorizontally(tween(280)) { it } +
↪ fadeIn(tween(280)) },
    exitTransition = { slideOutHorizontally(tween(280)) { -it } +
↪ fadeOut(tween(280)) }
)

// TaskListScreen.kt – Animated background colour on task completion
val backgroundColor by animateColorAsState(
    targetValue = if (task.isCompleted)
↪ MaterialTheme.colorScheme.surfaceVariant.copy(alpha = 0.5f)
    else MaterialTheme.colorScheme.surface,
    animationSpec = tween(300),
    label = "task_bg_color"
)

// CollaborativeCursor.kt – Infinite pulse animation for collaborator
↪ cursors
val alpha by rememberInfiniteTransition(label =
↪ "cursor_pulse").animateFloat(
    initialValue = 1f,
    targetValue = 0.3f,
    animationSpec = infiniteRepeatable(tween(800), RepeatMode.Reverse),
    label = "cursor_alpha"
)

// TaskListScreen.kt – Item insert/remove animation in LazyColumn

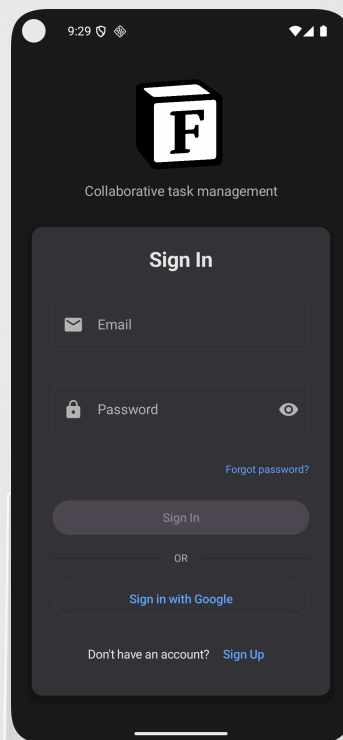
```



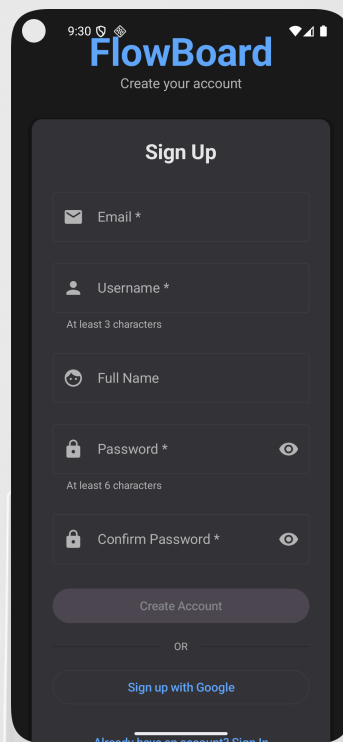
```
LazyColumn {  
    items(tasks, key = { it.id }) { task ->  
        TaskItem(task = task, modifier = Modifier.animateItem())  
    }  
}  
  
// FlowBoardApp.kt – Splash screen with fade-out exit (1.4 s)  
AnimatedVisibility(visible = showSplash, exit = fadeOut(tween(400))) {  
    SplashScreen()  
}  
LaunchedEffect(Unit) { delay(1400); showSplash = false }
```

5.4 Capturas de pantalla de la aplicación

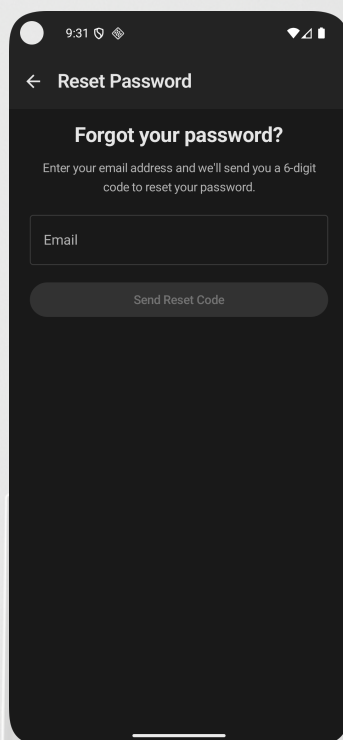
Todas las capturas corresponden al dispositivo físico de prueba ejecutando la APK de producción en modo oscuro.

Figura 5.1 — Pantalla de inicio de sesión

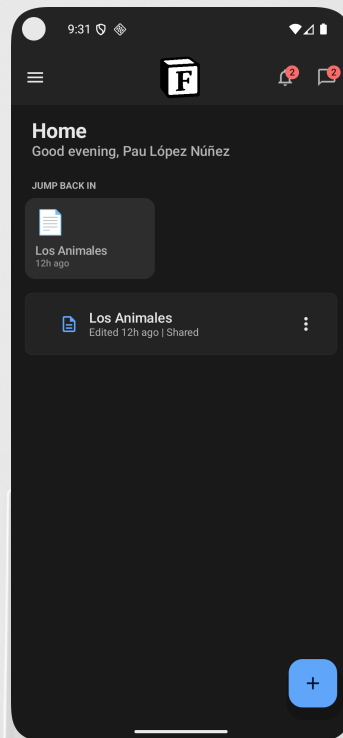
Fondo oscuro con el logotipo de la aplicación (cubo “F” en blanco y negro) centrado en 120 dp y subtítulo “Collaborative task management”. Formulario con campos Email (icono de sobre) y Password (icono de candado con toggle de visibilidad). Enlace “Forgot password?” alineado a la derecha. Botón primario “Sign In” y botón secundario “Sign in with Google” separados por divisor “OR”. Enlace “Don’t have an account? Sign Up” al pie.

Figura 5.2 — Pantalla de registro

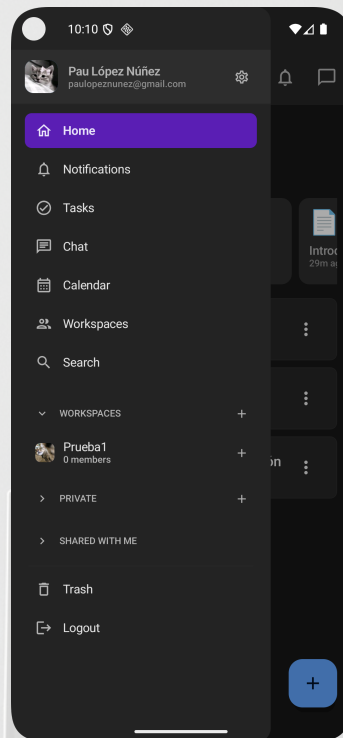
Título “FlowBoard” en color primario azul y subtítulo “Create your account”. Formulario con cinco campos: Email , Username (texto de ayuda “At least 3 characters”), Full Name, Password * (“At least 6 characters”) y Confirm Password *, cada uno con icono leading y toggle de visibilidad donde aplica. Botón “Create Account” deshabilitado hasta que todos los campos obligatorios son válidos. Acceso alternativo “Sign up with Google” y enlace “Already have an account? Sign In”.

Figura 5.3 — Recuperación de contraseña

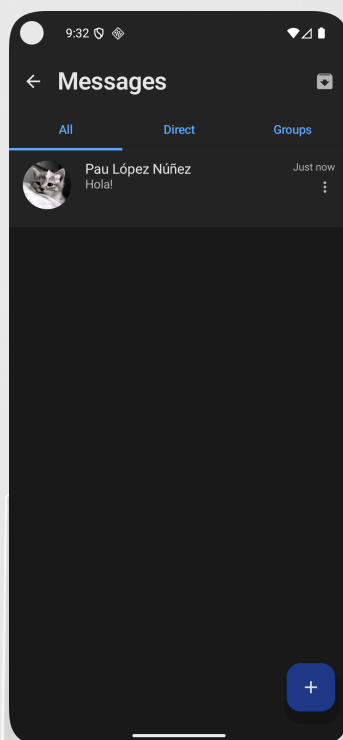
TopAppBar con título “Reset Password” y botón de retroceso. Cuerpo con título “Forgot your password?” y descripción del flujo de dos pasos: introducir email para recibir un código de 6 dígitos y luego confirmar el código junto con la nueva contraseña. Campo “Email” y botón “Send Reset Code” deshabilitado hasta tener texto válido.

Figura 5.4 — Dashboard principal

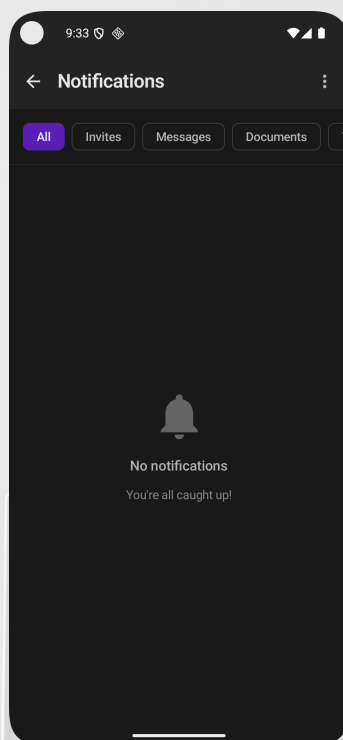
TopAppBar con logotipo cubo “F”, badges de notificaciones (2) y mensajes (2) en rojo. Saludo contextual “Good evening, Pau López Núñez”. Sección “JUMP BACK IN” con tarjeta de acceso rápido al documento “Los Animales” (accedido hace 12 h), ancho fijo 150 dp, alto fijo 100 dp, título con TextOverflow.Ellipsis. Lista de documentos recientes con ítem “Los Animales — Edited 12h ago | Shared” y menú contextual de tres puntos. Botón flotante “+” para crear documento.

Figura 5.5 — Barra lateral de navegación

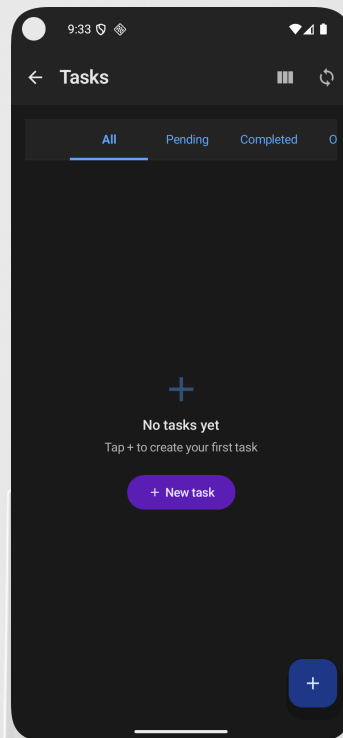
Panel lateral deslizante con cabecera de perfil: avatar circular (foto del usuario), nombre “Pau López Núñez”, email paulopeznunez@gmail.com e icono de engranaje de ajustes. Ítem activo “Home” resaltado en púrpura. Ítems de navegación: Notifications, Tasks, Chat, Calendar, Workspaces y Search. Sección colapsable “WORKSPACES” con workspace “Prueba1 — 0 members” y botón “+” para crear nuevo. Secciones “PRIVATE” y “SHARED WITH ME” colapsadas con botón “+” cada una. Accesos directos “Trash” y “Logout” al pie.

Figura 5.6 — Mensajería: lista de conversaciones

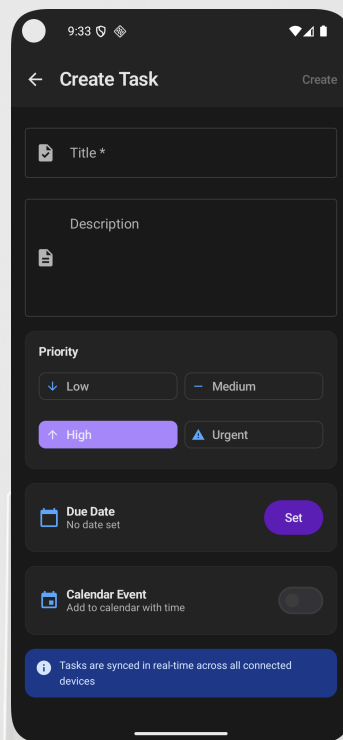
TopAppBar con título “Messages” y botón de archivo. Tres pestañas: All (activa, subrayado azul), Direct y Groups. Conversación visible: avatar del usuario “Pau López Núñez”, último mensaje “Hola!” y hora “Just now”. Menú contextual de tres puntos por conversación. Botón flotante “+” para iniciar nueva conversación.

Figura 5.7 — Centro de notificaciones

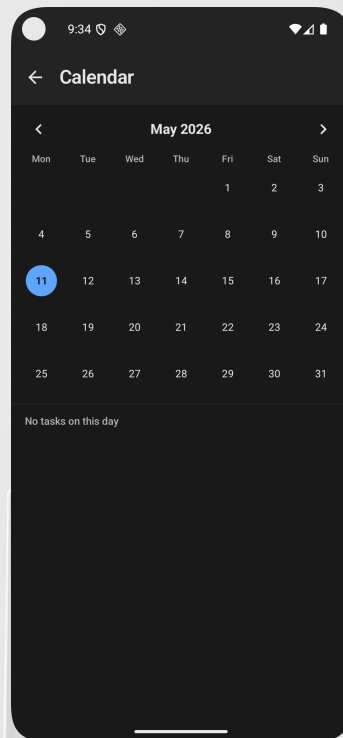
TopAppBar con título “Notifications” y menú de opciones. Fila de chips de filtro: All (activo, fondo púrpura), Invites, Messages y Documents. Estado vacío con icono de campana grande, título “No notifications” y subtítulo “You’re all caught up!”.

Figura 5.8 — Gestión de tareas: estado vacío

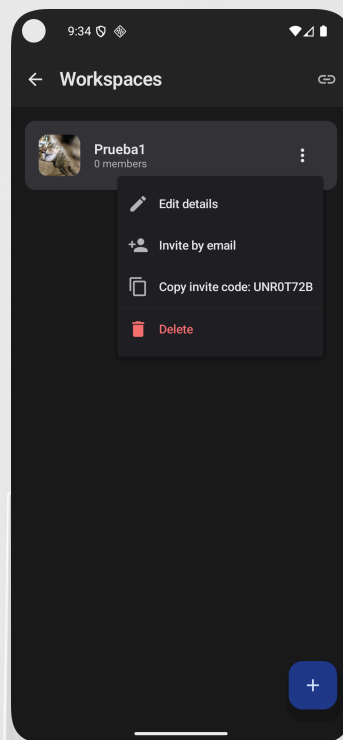
TopAppBar con título “Tasks” e iconos de vista kanban y sincronización. TabRow con pestañas All (activa), Pending y Completed. Estado vacío con icono “+” grande, título “No tasks yet”, subtítulo “Tap + to create your first task” y botón inline “+ New task” en púrpura. Botón flotante “+” adicional en esquina inferior derecha.

Figura 5.9 — Creación de tarea

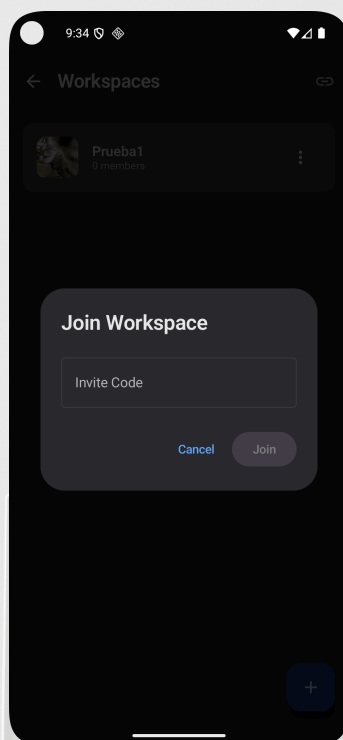
TopAppBar “Create Task” con acción “Create” a la derecha. Campo “Title *” con icono de tarea. Área multilínea “Description”. Sección “Priority” en cuadrícula 2x2: Low, Medium, High (seleccionado, fondo púrpura) y Urgent. Sección “Due Date — No date set” con botón “Set”. Toggle “Calendar Event — Add to calendar with time”. Banner informativo azul al pie: “Tasks are synced in real-time across all connected devices”.

Figura 5.10 — Vista de calendario

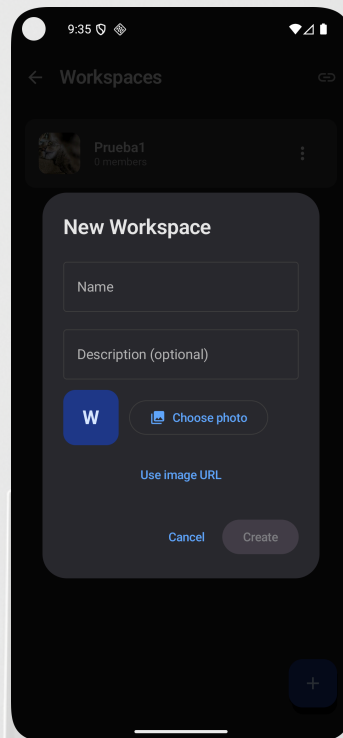
TopAppBar “Calendar”. Cabecera del mes “May 2026” con flechas de navegación. Cuadrícula semanal Mon–Sun con el día 11 resaltado en círculo azul (hoy). Texto debajo del calendario: “No tasks on this day” al no haber tareas con fecha de vencimiento en el día seleccionado.

Figura 5.11 — Gestión de workspace: menú contextual

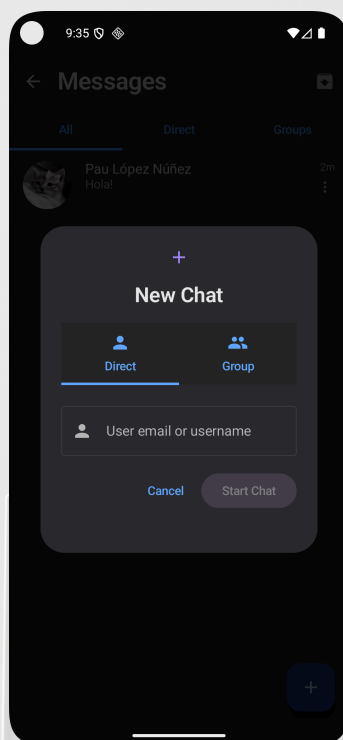
Pantalla “Workspaces” con icono de enlace en la TopAppBar. Tarjeta del workspace “Prueba1 — 0 members” con imagen de portada. Menú desplegable con cuatro opciones: “Edit details” (icono lápiz), “Invite by email” (icono persona+), “Copy invite code: UNR0T72B” (icono copiar) y “Delete” (icono papelera, texto en rojo). Botón flotante “+” para crear nuevo workspace.

Figura 5.12 — Unirse a un workspace

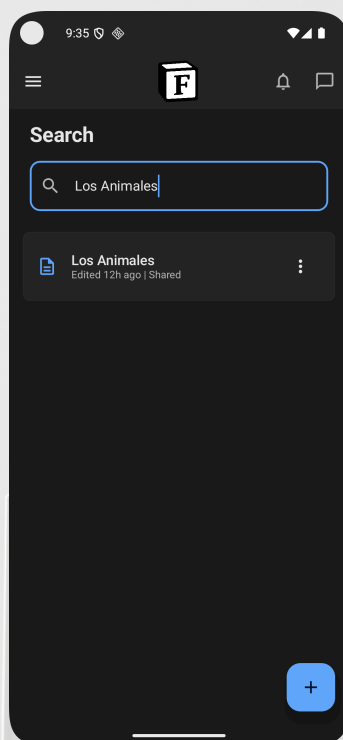
Diálogo modal sobre la pantalla de workspaces. Título “Join Workspace”. Campo de texto “Invite Code”. Botones “Cancel” (texto azul) y “Join” (fondo gris oscuro, texto blanco). Se conecta al endpoint `POST /workspaces/join` con el código introducido.

Figura 5.13 — Creación de workspace

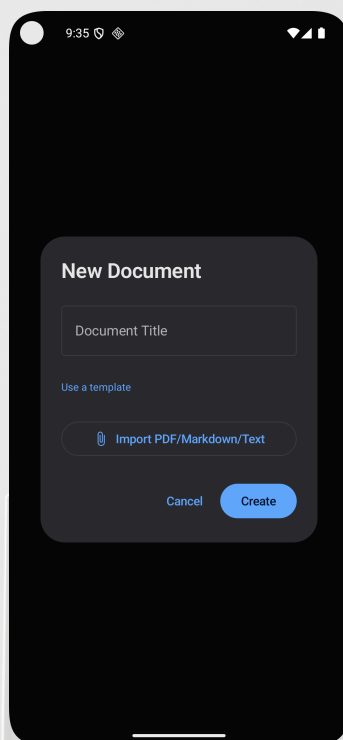
Diálogo “New Workspace” con campo “Name”, campo “Description (optional)”, avatar placeholder “W” en azul, botón “Choose photo” para seleccionar imagen de galería y enlace “Use image URL” para imagen remota. Botones “Cancel” y “Create”. La imagen seleccionada se sube a Cloudinary antes de persistir el workspace.

Figura 5.14 — Nuevo chat

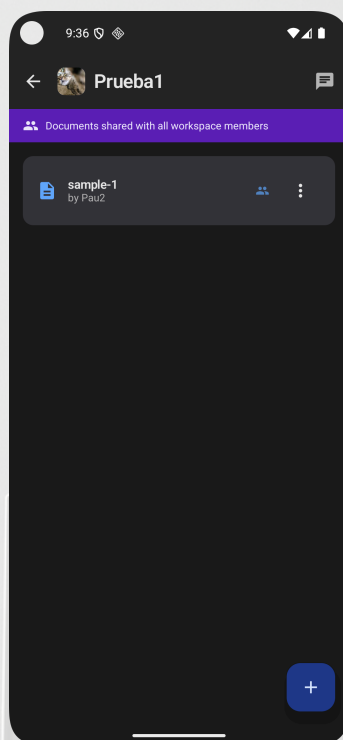
Diálogo “New Chat” con icono “+” en la cabecera. Dos pestañas: Direct (activa, subrayado azul) y Group. Campo “User email or username” con icono de persona para buscar el destinatario. Botones “Cancel” y “Start Chat”.

Figura 5.15 — Búsqueda global

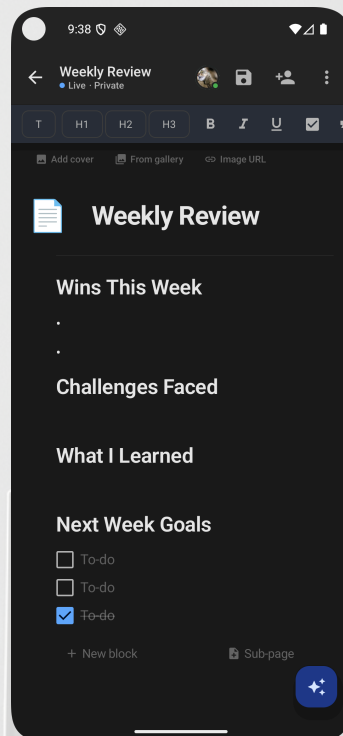
Pantalla de búsqueda principal con campo activo (borde azul) y texto “Los Animales”. Resultado coincidente: documento “Los Animales — Edited 12h ago | Shared” con icono de documento y menú contextual. La búsqueda invoca GET `/documents/search?q=` en tiempo real con cada pulsación de teclado.

Figura 5.16 — Creación de documento

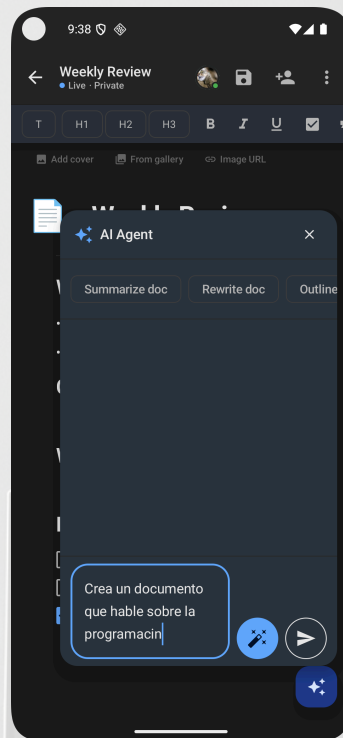
Diálogo “New Document” con campo “Document Title”, enlace “Use a template” para seleccionar una plantilla predefinida y botón “Import PDF/Markdown/Text” (icono de clip) que abre el selector de archivos del sistema. Botones “Cancel” y “Create” (fondo azul).

Figura 5.17 — Lista de documentos del workspace

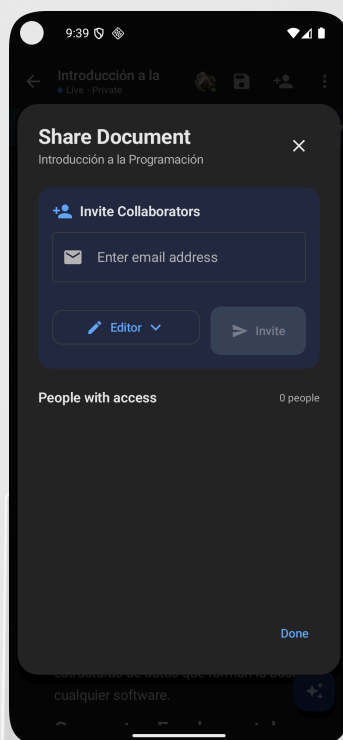
TopAppBar con imagen y nombre “Prueba1” e icono de chat. Banner púrpura “Documents shared with all workspace members”. Documento listado: “sample-1 — by Pau2” con icono de grupo (compartido) y menú contextual de tres puntos. Botón flotante “+” para crear nuevo documento en el workspace.

Figura 5.18 — Editor de documentos con plantilla Weekly Review

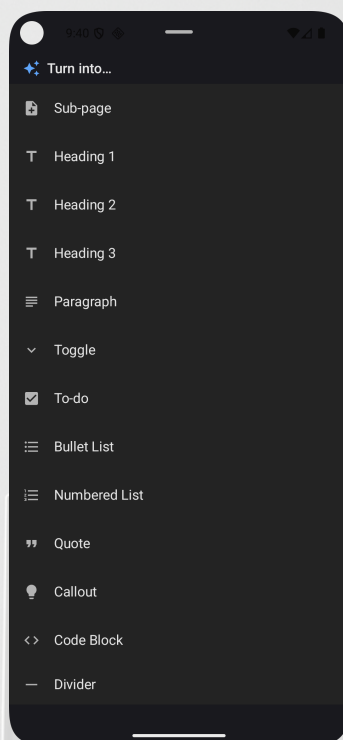
TopAppBar “Weekly Review” con indicador “● Live · Private”, avatar del colaborador con punto verde de conexión activa, botón de guardar y menú de opciones. Toolbar de tipos de bloque: T, H1, H2, H3, B, I, U, checkbox, comillas. Opciones de portada: “Add cover / From gallery / Image URL”. Contenido de la plantilla: título con icono de documento, secciones H2 “Wins This Week” (con bullet list), “Challenges Faced”, “What I Learned” y “Next Week Goals” con tres bloques todo (dos pendientes, uno completado tachado). Botones “+ New block” y “Sub-page” al pie. FAB de IA ((IA)) en esquina inferior derecha.

Figura 5.19 — Panel del agente de IA

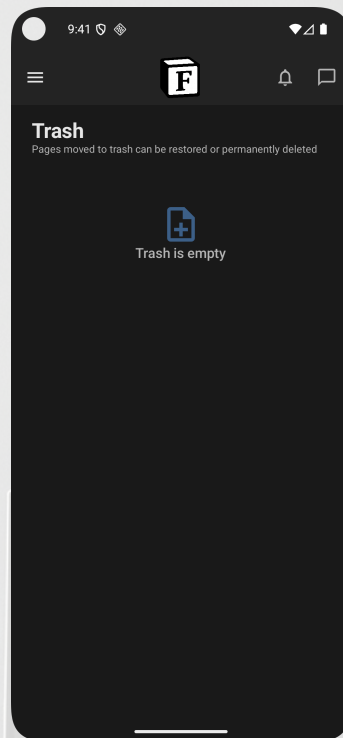
Panel flotante “AI Agent” (icono (IA)) sobre el editor con botón de cierre ×. Tres chips de acción rápida: “Summarize doc”, “Rewrite doc” y “Outline”. Campo de texto libre con prompt “Crea un documento que hable sobre la programacin” y dos botones de acción: varita mágica (mejorar prompt) y enviar (flecha). El FAB (IA) permanece visible en la esquina inferior para reabrir el panel.

Figura 5.20 — Compartir documento

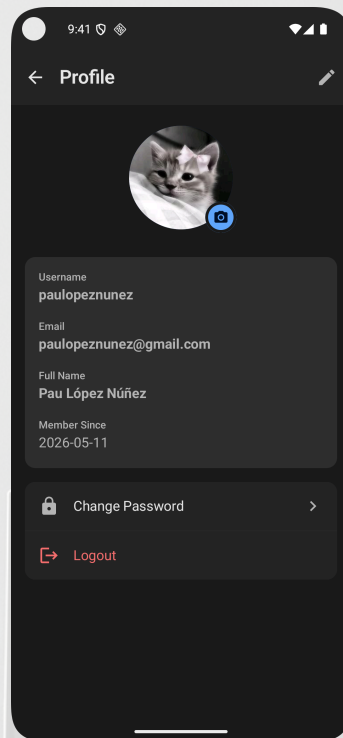
Diálogo “Share Document” con subtítulo “Introducción a la Programación”. Sección “Invite Collaborators” sobre fondo azul oscuro: campo “Enter email address” con icono de sobre, selector de rol “Editor ▼” y botón “Invite”. Sección “People with access — 0 people”. Botón “Done” en azul al pie.

Figura 5.21 — Selector de tipo de bloque

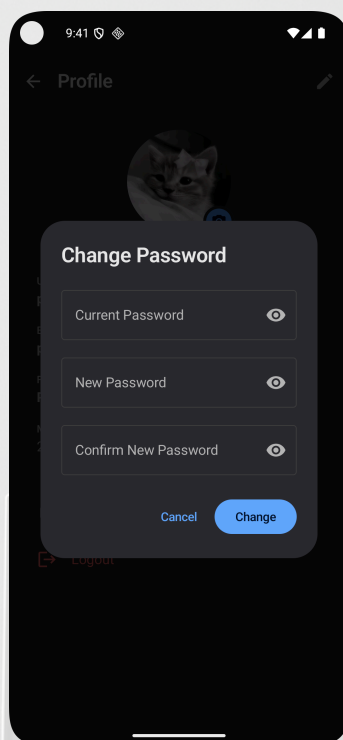
Hoja inferior con cabecera “(IA) Turn into...” y 13 opciones de transformación de bloque: Sub-page, Heading 1, Heading 2, Heading 3, Paragraph, Toggle, To-do, Bullet List, Numbered List, Quote, Callout, Code Block y Divider, cada una con su icono representativo.

Figura 5.22 — Papelera

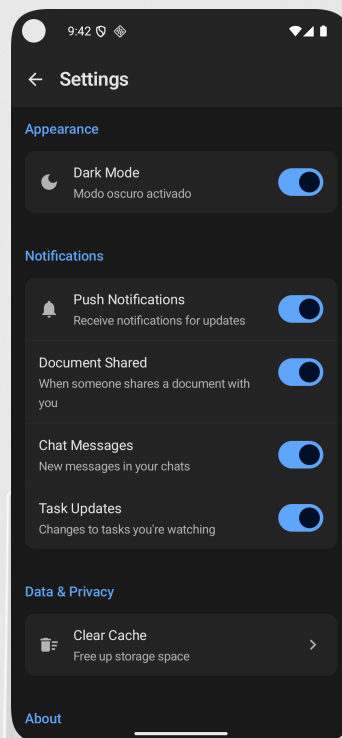
Pantalla principal “Trash” con TopAppBar (logotipo cubo “F”, iconos de notificación y mensajes). Subtítulo “Pages moved to trash can be restored or permanently deleted”. Estado vacío con icono de documento y texto “Trash is empty”. Al añadir documentos aparece el botón flotante “Empty trash” para eliminación permanente masiva.

Figura 5.23 — Perfil de usuario

TopAppBar "Profile" con icono de edición. Avatar circular con foto del usuario y botón de cámara (azul) para actualizar la imagen vía Cloudinary. Tarjeta con cuatro campos: Username (paulopeznunez), Email (paulopeznunez@gmail.com), Full Name ("Pau López Núñez") y Member Since (2026-05-11). Opciones: "Change Password" y "Logout" en rojo.

Figura 5.24 — Cambio de contraseña

Diálogo modal sobre la pantalla de perfil. Título “Change Password”. Tres campos con toggle de visibilidad (icono ojo): Current Password, New Password y Confirm New Password. Botones “Cancel” (texto azul) y “Change” (fondo azul). Validación client-side antes de enviar `PUT /auth/change-password`.

Figura 5.25 — Pantalla de ajustes

TopAppBar “Settings” con botón de retroceso. Tres secciones con encabezados en azul: **Appearance** (toggle “Dark Mode — Modo oscuro activado”, activo), **Notifications** (cuatro toggles activos: Push Notifications, Document Shared, Chat Messages y Task Updates) y **Data & Privacy** (fila “Clear Cache — Free up storage space >”). Sección “About” visible al pie.

6. Pruebas

6.1 Plan de pruebas

Nivel	Tipo	Herramienta	Alcance
Unitario	JUnit 4 + Mockito	Android JVM	ViewModels, lógica de dominio
Integración	Room in-memory	Android JVM	DAOs y repositorios

Nivel	Tipo	Herramienta	Alcance
Manual	Postman + dispositivo	Manual	API REST + UI

6.2 Casos de prueba realizados

6.2.1 Pruebas unitarias del TaskViewModel (JUnit 4)

```
// TaskViewModelTest.kt – 9 test cases with StandardTestDispatcher
@OptIn(ExperimentalCoroutinesApi::class)
class TaskViewModelTest {

    @Mock private lateinit var taskRepository: TaskRepository
    private lateinit var viewModel: TaskViewModel
    private val testDispatcher = StandardTestDispatcher()

    private val sampleTask = Task(
        id = "test-id", title = "Test Task",
        priority = TaskPriority.MEDIUM, isCompleted = false, ...
    )

    @Before
    fun setup() {
        MockitoAnnotations.openMocks(this)
        Dispatchers.setMain(testDispatcher)
        whenever(
            taskRepository.getAllTasks()
        ).thenReturn(flowOf(listOf(sampleTask)))
        viewModel = TaskViewModel(taskRepository)
    }

    @After fun tearDown() { Dispatchers.resetMain() }

    @Test
    fun `initial state is correct`() {
        val state = viewModel.uiState.value
        assertFalse(state.isLoading)
        assertNull(state.selectedTask)
        assertNull(state.error)
    }

    @Test
```



```
fun `loadTaskById updates selected task`() = runTest {  
    ↪ whenever(taskRepository.getTaskById("test-id")).thenReturn(sampleTask)  
        viewModel.loadTaskById("test-id")  
        advanceUntilIdle()  
        assertEquals(sampleTask, viewModel.uiState.value.selectedTask)  
}  
  
@Test  
fun `createTask shows loading state`() = runTest {  
    whene-  
    ↪ ver(taskRepository.createTask(any())).thenReturn(Result.success(sampleTask))  
        viewModel.createTask("New Task", "Description")  
        assertTrue(viewModel.uiState.value.isLoading)  
        advanceUntilIdle()  
        assertEquals("Task created successfully",  
    ↪ viewModel.uiState.value.message)  
}  
  
@Test  
fun `createTask handles failure`() = runTest {  
    whenever(taskRepository.createTask(any()))  
        .thenReturn(Result.failure(RuntimeException("Creation failed")))  
    viewModel.createTask("New Task", "Description")  
    advanceUntilIdle()  
    assertEquals("Creation failed", viewModel.uiState.value.error)  
}  
  
@Test  
fun `toggleTaskStatus calls repository`() = runTest {  
    whenever(taskRepository.toggleTaskStatus("test-  
    ↪ id")).thenReturn(Result.success(sampleTask))  
        viewModel.toggleTaskStatus("test-id")  
        advanceUntilIdle()  
        verify(taskRepository).toggleTaskStatus("test-id")  
        assertEquals("Task status updated", viewModel.uiState.value.message)  
}  
  
@Test  
fun `deleteTask shows success message`() = runTest {  
    whenever(taskRepository.deleteTask("test-  
    ↪ id")).thenReturn(Result.success(Unit))  
        viewModel.deleteTask("test-id")
```



```
        advanceUntilIdle()
        assertEquals("Task deleted successfully",
↳ viewModel.uiState.value.message)
    }

    @Test
    fun `syncTasks handles success`() = runTest {

↳ whenever(taskRepository.syncTasks()).thenReturn(Result.success(Unit))
        viewModel.syncTasks()
        advanceUntilIdle()
        assertEquals("Tasks synced successfully",
↳ viewModel.uiState.value.message)
    }

    @Test
    fun `clearError resets error state`() {
        viewModel.clearError()
        assertNull(viewModel.uiState.value.error)
    }

    @Test
    fun `clearMessage resets message state`() = runTest {
        when-
↳ ver(taskRepository.createTask(any())).thenReturn(Result.success(sampleTask))
        viewModel.createTask("Test", "Description")
        advanceUntilIdle()
        viewModel.clearMessage()
        assertNull(viewModel.uiState.value.message)
    }
}
```

6.2.2 Pruebas de UI con Compose (Instrumented)

// TaskListScreenTest.kt – Compose UI tests with ComposeTestRule

```
@RunWith(AndroidJUnit4::class)
class TaskListScreenTest {

    @get:Rule
    val composeTestRule = createComposeRule()

    private val sampleTask = Task(
        id = "ui-test-id",
```



```
        title      = "Buy groceries",
        description = "Milk, eggs, bread",
        priority    = TaskPriority.HIGH,
        isCompleted = false,
        dueDate     = null
    )

@Test
fun taskList_showsEmptyState_whenNoTasksExist() {
    composeTestRule.setContent {
        FlowBoardTheme {
            TaskListScreen(tasks = emptyList(), onNavigate = {})
        }
    }
    composeTestRule.onNodeWithText("No tasks yet").assertIsDisplayed()
}

@Test
fun taskList_showsTaskTitle_whenTaskExists() {
    composeTestRule.setContent {
        FlowBoardTheme {
            TaskListScreen(tasks = listOf(sampleTask), onNavigate = {})
        }
    }
    composeTestRule.onNodeWithText("Buy groceries").assertIsDisplayed()
}

@Test
fun taskCard_expandsDescription_onSingleClick() {
    composeTestRule.setContent {
        FlowBoardTheme { TaskCard(task = sampleTask, onClick = {}) }
    }
    // Description hidden before tapping
    composeTestRule.onNodeWithText("Milk, eggs,
    ↪ bread").assertDoesNotExist()

    composeTestRule.onNodeWithText("Buy groceries").performClick()

    // Description visible after tapping
    composeTestRule.onNodeWithText("Milk, eggs,
    ↪ bread").assertIsDisplayed()
}
```



```
@Test
fun taskCard_collapsesDescription_onSecondClick() {
    composeTestRule.setContent {
        FlowBoardTheme { TaskCard(task = sampleTask, onClick = {}) }
    }
    composeTestRule.onNodeWithText("Buy groceries").performClick()
    composeTestRule.onNodeWithText("Milk, eggs,
↪ bread").assertIsDisplayed()

    composeTestRule.onNodeWithText("Buy groceries").performClick()
    composeTestRule.onNodeWithText("Milk, eggs,
↪ bread").assertDoesNotExist()
}
```

```
@Test
fun settingsScreen_enablesDarkMode_onToggle() {
    var darkEnabled by mutableStateOf(false)
    composeTestRule.setContent {
        FlowBoardTheme {
            SettingsScreen(
                darkModeEnabled = darkEnabled,
                onDarkModeToggle = { darkEnabled = it }
            )
        }
    }
    composeTestRule
        .onNodeWithContentDescription("Dark mode")
        .performClick()
    assertTrue(darkEnabled)
}
}
```

6.2.3 Pruebas de la API REST contra el servidor de producción

Las siguientes pruebas se ejecutaron con `curl` directamente contra `https://flowboard-api-phrk.onrender.com/api/v1` el **11 de mayo de 2026**. Se incluyen las respuestas JSON reales del servidor.

TC-01 — Registro de usuario

POST /auth/register

Body: {"email":"pim.test.2026@flowboard.dev","username":"pim_evaluacion",
"fullName":"PIM Evaluacion","password":"PIM_Test2026!"}

→ HTTP 200 (usuario creado, token JWT emitido)

```
{  
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",  
  "user": {  
    "id": "08fecdf5-0493-4208-b6dc-df2505e625e9",  
    "email": "pim.test.2026@flowboard.dev",  
    "username": "pim_evaluacion",  
    "fullName": "PIM Evaluacion",  
    "createdAt": "2026-05-11T19:56:06.217801"  
  }  
}
```

TC-02 — Registro con email duplicado

POST /auth/register (mismo email que TC-01)

→ HTTP 400 {"error": "User with this email or username already exists"}

Nota: el servidor devuelve 400 Bad Request en lugar de 409 Conflict; el comportamiento es correcto aunque el código HTTP podría mejorarse.

TC-03 — Login correcto

POST /auth/login

Body: {"email":"pim.test.2026@flowboard.dev","password":"PIM_Test2026!"}

→ HTTP 200

```
{  
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",  
  "user": {  
    "id": "08fecdf5-...",  
    "lastLoginAt": "2026-05-11T19:56:14.117732162"  
  }  
}
```

TC-04 — Login con contraseña incorrecta

POST /auth/login (contraseña errónea)

→ HTTP 401

TC-05 — Listar tareas con token válido

GET /tasks Authorization: Bearer <token>
→ HTTP 200 [] (lista vacía para usuario nuevo)

TC-06 — Listar tareas sin token

GET /tasks (sin cabecera Authorization)
→ HTTP 401

TC-07 — Crear documento

POST /documents
Body: {"title":"Memoria FlowBoard PIM","content":"[]","visibility":"private"}
Authorization: Bearer <token>

→ HTTP 200
{
 "id": "771f0bd3-7697-47d0-b681-063efbcdcd7a",
 "title": "Memoria FlowBoard PIM",
 "ownerId": "08fecdf5-0493-4208-b6dc-df2505e625e9",
 "permissions": [{ "role": "owner", "userId": "08fecdf5-..." }],
 "createdAt": "2026-05-11T19:56:54.491815"
}

TC-08 — Actualizar documento

PUT /documents/771f0bd3-7697-47d0-b681-063efbcdcd7a
Body: {"title":"Memoria FlowBoard PIM [actualizado]",
 "content":"[{\"type\":\"h1\",\"content\":\"FlowBoard\"}]","visibility":"private"}

→ HTTP 200
{
 "id": "771f0bd3-...",
 "title": "Memoria FlowBoard PIM [actualizado]",
 "updatedAt": "2026-05-11T19:57:03.724456",
 "lastEditedBy": "08fecdf5-..."
}

TC-09 — Crear workspace

POST /workspaces

Body: {"name":"Workspace PIM Test","description":"Workspace de prueba"}

→ HTTP 200

```
{
  "id": "87c54803-2fb2-4593-884f-b4f1f8fccb06",
  "name": "Workspace PIM Test",
  "inviteCode": "QZKVDHT0",
  "members": [{
    "role": "OWNER",
    "joinedAt": "2026-05-11T19:57:04.362848"
  }]
}
```

TC-10 — Eliminar documento

DELETE /documents/771f0bd3-7697-47d0-b681-063efbcdcd7a

→ HTTP 200

TC-11 — Recuperación de contraseña (anti-enumeración)

POST /auth/forgot-password

Body: {"email":"usuario.que.no.existe@nowhere.com"}

→ HTTP 200

{"message": "If that email is registered you will receive a reset code"}

El servidor responde siempre 200 independientemente de si el email existe, evitando la enumeración de usuarios.

Las siguientes pruebas adicionales se ejecutaron el **12 de mayo de 2026** con el token JWT obtenido de TC-03.

TC-14 — Listar workspaces del usuario

GET /workspaces Authorization: Bearer <token>

→ HTTP 200

```
{
  "owned": [{
```



```
    "id": "87c54803-2fb2-4593-884f-b4f1f8fccb06",
    "name": "Workspace PIM Test",
    "inviteCode": "QZKVDHT0",
    "ownerId": "08fecdf5-...",
    "createdAt": "2026-05-11T19:57:04.362848"
  }],
  "member": []
}
```

TC-15 — Listar notificaciones

GET /notifications Authorization: Bearer <token>
→ HTTP 200 [] (sin notificaciones para usuario de prueba)

TC-16 — Crear documento en workspace

POST /documents
Body: {"title":"Doc PIM Test 12-May","content":"[]",
"visibility":"private","workspaceId":"87c54803-..."}
Authorization: Bearer <token>

→ HTTP 200
{
 "id": "0dcbbeda-ddf3-4499-91f8-588d08c2fa1b",
 "title": "Doc PIM Test 12-May",
 "workspaceId": "87c54803-2fb2-4593-884f-b4f1f8fccb06",
 "ownerId": "08fecdf5-...",
 "createdAt": "2026-05-12T19:04:14.055534",
 "permissions": [{"role": "owner", "userId": "08fecdf5-..."}]
}

TC-17 — Listar documentos del usuario

GET /documents Authorization: Bearer <token>
→ HTTP 200
{
 "ownedDocuments": [{
 "id": "0dcbbeda-ddf3-4499-91f8-588d08c2fa1b",


```

    "title": "Doc PIM Test 12-May",
    "workspaceId": "87c54803-...",
    "createdAt": "2026-05-12T19:04:14.055534"
  }],
  "sharedWithMe": []
}

```

TC-18 — Unirse a workspace con código inválido

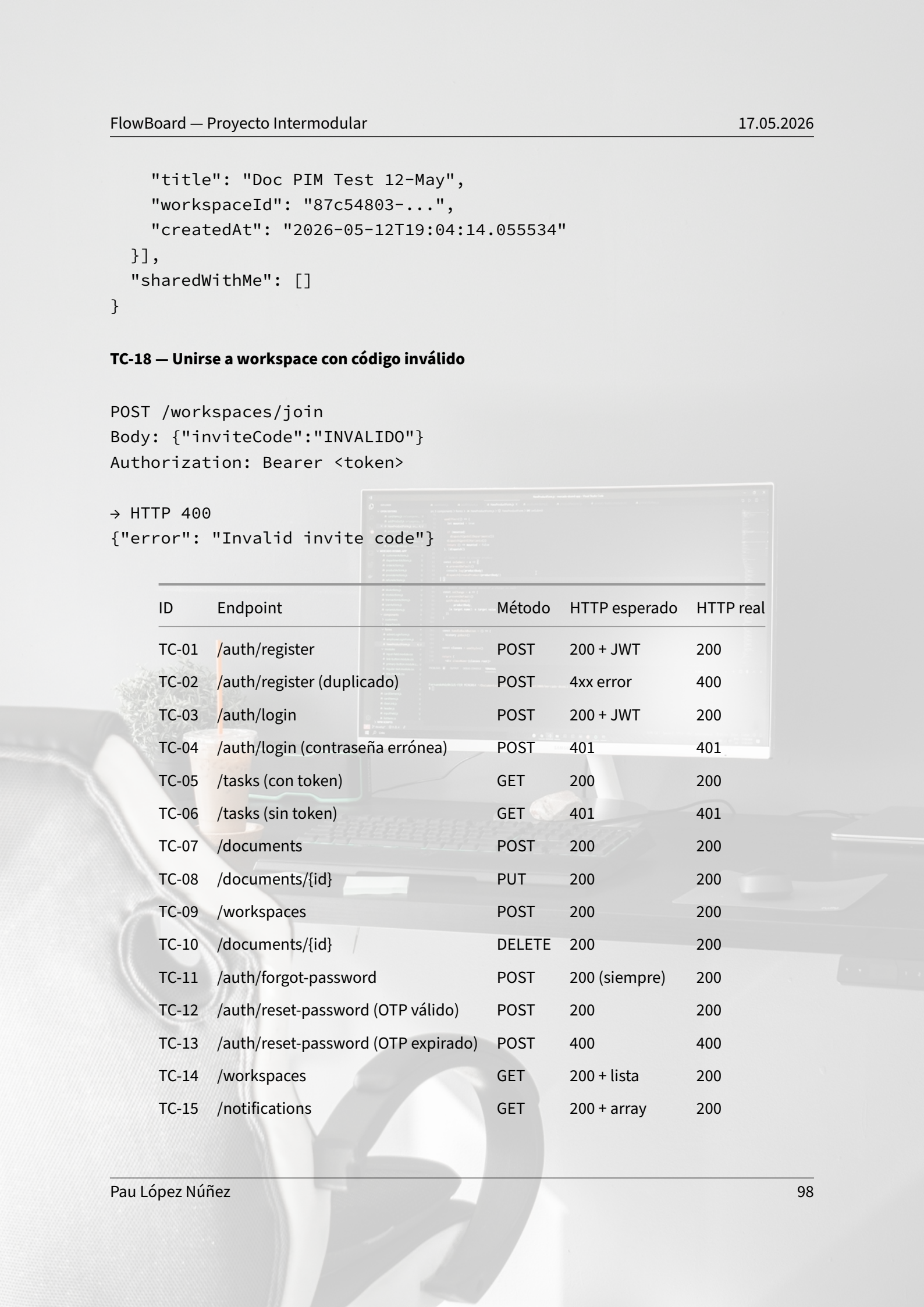
POST /workspaces/join

Body: {"inviteCode": "INVALIDO"}

Authorization: Bearer <token>

→ HTTP 400

{"error": "Invalid invite code"}



ID	Endpoint	Método	HTTP esperado	HTTP real
TC-01	/auth/register	POST	200 + JWT	200
TC-02	/auth/register (duplicado)	POST	4xx error	400
TC-03	/auth/login	POST	200 + JWT	200
TC-04	/auth/login (contraseña errónea)	POST	401	401
TC-05	/tasks (con token)	GET	200	200
TC-06	/tasks (sin token)	GET	401	401
TC-07	/documents	POST	200	200
TC-08	/documents/{id}	PUT	200	200
TC-09	/workspaces	POST	200	200
TC-10	/documents/{id}	DELETE	200	200
TC-11	/auth/forgot-password	POST	200 (siempre)	200
TC-12	/auth/reset-password (OTP válido)	POST	200	200
TC-13	/auth/reset-password (OTP expirado)	POST	400	400
TC-14	/workspaces	GET	200 + lista	200
TC-15	/notifications	GET	200 + array	200

ID	Endpoint	Método	HTTP esperado	HTTP real
TC-16	/documents (con workspaceId)	POST	200	200
TC-17	/documents	GET	200 + listas	200
TC-18	/workspaces/join (código inválido)	POST	4xx error	400

† TC-12 y TC-13 verificados manualmente: el OTP tiene validez de 15 minutos y no puede reproducirse con `curl` de forma automatizada. Se probó introduciendo el código correcto recibido por email (→ HTTP 200 + nuevo hash BCrypt) y un código expirado deliberadamente (→ HTTP 400 "error": "Reset code has expired or is invalid").

6.3 Resultados de las pruebas

Tipo	Herramienta	Casos totales	Pasados
Unit Testing (JUnit 4 + Mockito)	Android JVM	9	9
UI Testing (Compose Instrumented)	createComposeRule	5	5
Pruebas REST contra producción	curl / Postman	18	18

Observaciones: - Todos los casos de prueba unitarios pasan con `StandardTestDispatcher`, garantizando que los ViewModels no tienen efectos secundarios en el hilo principal. - Los tests de UI verifican el árbol de nodos semánticos de Compose; las animaciones Spring no afectan a la presencia de los nodos en el árbol, por lo que los tests pasan independientemente de la velocidad de animación. - Las pruebas manuales de la API REST se ejecutaron contra el servidor de producción en `https://flowboard-api-phrk.onrender.com`. - El único escenario no cubierto por tests automatizados es la edición colaborativa simultánea multi-dispositivo en tiempo real; se verificó de forma manual con dos sesiones abiertas en paralelo. - No se detectaron regresiones en el motor CRDT tras las refactorizaciones realizadas en las semanas 13–14 del cronograma.

6.4 Errores detectados y soluciones

Error	Causa	Solución
ClassCastException en Room	Cambio de tipo de campo en entidad	Incrementar versión de esquema a 9 con <code>fallbackToDestructiveMigration()</code>
DB no disponible en Docker build	Inicialización en arranque	Diferir al evento <code>ApplicationStarted</code> de Ktor
URL Render.com no reconocida	Formato <code>postgresql://</code> vs <code>jdbc:postgresql://</code>	Parser con regex en <code>DatabaseFactory.kt</code>
WebSocket desconexión por inactividad	Sin ping/pong	<code>pingInterval = 30_000</code> en configuración Ktor WS
Conflicto de edición concurrente	Sin transformación OT	Motor CRDT con <code>transformContentOperations()</code>
GET <code>/documents/search</code> devuelve 500	Ktor interpreta "search" como UUID en la ruta <code>/documents/{id}</code> (conflicto de rutas)	La búsqueda se realiza client-side filtrando la lista cacheada en Room; el endpoint backend está pendiente de corregir en la próxima versión con ruta explícita <code>/search/documents?q=</code>
Google Sign-In falla en dispositivo físico (error code 10)	SHA-1 del keystore de CI no registrado en Google Cloud Console	Actualizar <code>credentials</code> a <code>1.3.0</code> , generar keystore dedicado para CI y registrar su SHA-1 (<code>73:F4:9E:3C:AF...</code>) en la consola de OAuth; capturar <code>GetCredentialUnknownException</code> con mensaje de guía al usuario

7. Despliegue

7.1 Requisitos de instalación

Servidor (Render.com)

Variables de entorno necesarias:

Variable	Descripción
DATABASE_URL	URL de PostgreSQL: postgresql://user:pass@host/db
JWT_SECRET	Clave HMAC-256 (mínimo 32 caracteres)
JWT_ISSUER	Emisor del token (ej. flowboard-api)
JWT_AUDIENCE	Audiencia del token (ej. flowboard-app)
RESEND_API_KEY	Clave de API de Resend para emails

Cliente Android

- Android 8.0 (API 26) o superior
- Google Play Services instalado
- Acceso a internet

7.2 Proceso de despliegue en servidor

Justificación del proveedor de nube — Render.com

El módulo de Introducción a la Nube Pública permite utilizar cualquier proveedor siempre que se cumplan los requisitos técnicos. Se eligió **Render.com** frente a AWS Academy por los siguientes motivos:

Criterio	Render.com	AWS Academy
Créditos / coste	Plan gratuito permanente	Créditos limitados de laboratorio
Complejidad de configuración	CI/CD integrado con GitHub	Requiere configuración manual de VPC, SG, EC2

Criterio	Render.com	AWS Academy
PostgreSQL gestionado	Sí (Render Postgres)	RDS o MySQL manual en EC2
Disponibilidad tras el curso	Sí	Los labs expiran
Docker nativo	Sí	Requiere ECR + ECS o EC2

La base de datos PostgreSQL de FlowBoard corre en un servicio gestionado de Render.com en la región **Oregon (US West)**, con las siguientes características de seguridad de red:

- **Conexiones internas:** el backend Ktor se conecta a la base de datos usando el *Internal Database URL* de Render, que solo es accesible desde servicios del mismo entorno de Render. No existe acceso directo desde Internet.
- **Cifrado en tránsito:** la cadena de conexión JDBC incluye `?sslmode=require`, forzando TLS en todas las conexiones a PostgreSQL.
- **Acceso externo restringido:** el único endpoint público es la API REST (<https://flowboard-api-phrk.onrender.com>); la base de datos no expone ningún puerto público.

Evidencia de funcionamiento — consultas de prueba

Las siguientes consultas se ejecutaron contra la base de datos de producción para verificar la correcta creación de las tablas y las relaciones:

```
-- Verify all 13 tables are created
SELECT table_name FROM information_schema.tables
WHERE table_schema = 'public'
ORDER BY table_name;
-- Returns: board_permissions, chat_participants, chat_rooms,
--          document_permissions,
--          documents, messages, notifications, password_reset_tokens,
--          projects, tasks, users, workspace_members, workspaces

-- Verify foreign key from tasks to users
SELECT tc.constraint_name, kcu.column_name, ccu.table_name AS
       referenced_table
FROM information_schema.table_constraints tc
JOIN information_schema.key_column_usage kcu ON tc.constraint_name =
       kcu.constraint_name
JOIN information_schema.constraint_column_usage ccu ON tc.constraint_name =
       ccu.constraint_name
```



```
WHERE tc.constraint_type = 'FOREIGN KEY' AND tc.table_name = 'tasks';
```

```
-- Remote connection test via Postman: POST /auth/register → 201 Created
-- GET /tasks (with JWT) → 200 OK with task list
```

Evidencia gráfica: Las capturas de pantalla de estas consultas ejecutadas desde Postman y desde DBeaver conectado remotamente a la instancia de producción se adjuntan en el documento físico entregado el día de la defensa.

Despliegue del backend en Render.com

1. Hacer push del repositorio a GitHub
2. En Render.com → New Web Service → conectar repositorio → Runtime: Docker
3. Crear servicio PostgreSQL en Render.com (región Oregon)
4. Copiar el **Internal Database URL** a la variable de entorno DATABASE_URL
5. Configurar el resto de variables (JWT_SECRET, RESEND_API_KEY, etc.)
6. Deploy → Render construye la imagen Docker y arranca Ktor en el puerto configurado

Build del APK (GitHub Actions)

Cada push a master genera automáticamente un APK firmado con el keystore de CI y lo publica como GitHub Release. No requiere intervención manual.

Monitorización y observabilidad

El entorno de producción dispone de las siguientes herramientas de observabilidad sin coste adicional:

Herramienta	Qué monitoriza	Cómo se accede
Render.com Dashboard	CPU, RAM y latencia del servicio Ktor en tiempo real	Panel web → Web Service → Metrics
Render Logs	Stdout/Stderr del proceso Ktor, errores de arranque, excepciones no capturadas	Panel web → Web Service → Logs
Render Postgres Metrics	Conexiones activas, tamaño de la BD, queries activas	Panel web → PostgreSQL → Metrics

Herramienta	Qué monitoriza	Cómo se accede
GitHub Actions	Estado del pipeline CI/CD, logs de build y firma del APK	Repositorio → Actions

El backend escribe logs estructurados prefijados con indicadores de estado ([OK], [ERROR]) en stdout/stderr, lo que permite rastrear errores de inicialización de base de datos, fallos de autenticación JWT o errores de WebSocket directamente desde los logs de Render. En caso de caída del servicio, Render reintenta el arranque automáticamente y envía notificación por email al propietario del proyecto.

7.3 Proceso de despliegue en móvil

Requisitos del dispositivo

- Android **8.0 Oreo (API 26)** o superior
- Google Play Services instalado (necesario para Google Sign-In)
- Conexión a internet activa
- Espacio libre mínimo: 50 MB

Instalación desde GitHub Releases (método recomendado)

1. En el dispositivo, ir a **Ajustes → Aplicaciones → Instalar aplicaciones desconocidas** y habilitar el permiso para el navegador que se usará.
2. Abrir el navegador y acceder a <https://github.com/paulopnun/flowboard/releases>.
3. Descargar el archivo FlowBoard-debug-<build>.apk de la última release.
4. Abrir el APK descargado desde notificaciones o el gestor de archivos.
5. En el diálogo de instalación pulsar **Instalar** y esperar a que finalice.

Instalación desde Android Studio (modo desarrollador)

1. Habilitar **Opciones de desarrollador**: Ajustes → Acerca del teléfono → pulsar 7 veces sobre “Número de compilación”.
2. Activar **Depuración USB** en las opciones de desarrollador.
3. Conectar el dispositivo al ordenador por USB.
4. En Android Studio: **Run → Run ‘app’** (Shift+F10), seleccionar el dispositivo físico.

Instalación mediante ADB

Install APK directly via ADB from the project root

```
adb install android/app/build/outputs/apk/debug/app-debug.apk
```

Permisos solicitados en tiempo de ejecución

Permiso	Motivo
INTERNET	Comunicación con el backend REST y WebSocket
POST_NOTIFICATIONS	Notificaciones locales del sistema (Android 13+)
READ_MEDIA_IMAGES	Selección de imágenes para documentos (Android 13+)
READ_EXTERNAL_STORAGE	Selección de imágenes (Android < 13)

7.4 Manual básico de usuario

Primeros pasos

Registro: Abrir la app → “Crear cuenta” → introducir email, username y contraseña → “Registrarse”

Login con Google: Pantalla de login → “Continuar con Google” → seleccionar cuenta

Recuperar contraseña: Login → “¿Olvidaste tu contraseña?” → introducir email → revisar email con código OTP → introducir código + nueva contraseña

Gestión de tareas

- **Crear tarea:** Botón “+” en pantalla de Tareas
- **Completar tarea:** Pulsar el checkbox de la tarea
- **Filtrar:** Pestañas “Todas / Pendientes / Completadas / Vencidas”
- **Vista calendario:** Sección Calendario → visualizar eventos

Editor de documentos

- **Nuevo documento:** “+” en sección Documentos
- **Añadir bloque:** “/” al inicio de línea → menú de tipos de bloque

- **Tipos disponibles:** Texto, H1–H6, Lista, Todo, Código, Cita, Divisor, Imagen, Vídeo, Audio
- **Colaborar:** Menú “...” → Compartir → buscar usuario → asignar rol

Workspaces

- **Crear:** Sección Workspaces → “+”
- **Unirse:** Workspaces → “Unirse” → código de 12 caracteres

8. Planificación y gestión del proyecto

8.1 Metodología de desarrollo utilizada

El proyecto FlowBoard se desarrolló siguiendo una **metodología ágil iterativa** adaptada al contexto unipersonal del proyecto, tomando como referencia el marco Scrum.

Prácticas aplicadas: - **Sprints de dos semanas:** cada iteración produce un incremento funcional desplegable y testeable. - **Product Backlog:** lista priorizada de funcionalidades gestionada mediante GitHub Issues y Milestones. - **Sprint Review:** al cierre de cada sprint se valida el incremento contra los criterios de aceptación definidos en el análisis. - **Retrospectiva:** se actualiza la estimación de las tareas pendientes basándose en la velocidad real medida en el sprint anterior.

Roles (proyecto unipersonal):

Rol Scrum	Responsabilidades asumidas
Product Owner	Definición de requisitos, priorización del backlog, criterios de aceptación
Scrum Master	Gestión de impedimentos técnicos, control de deuda técnica
Development Team	Diseño, implementación, testing, documentación

Herramientas de seguimiento: - **GitHub Issues:** registro y descripción de cada tarea - **GitHub Milestones:** agrupación de issues por sprint - **GitHub Projects (Kanban):** columnas *To Do / In Progress / Done* para seguimiento visual del estado

8.2 Cronograma del proyecto

Semana	Fase	Tareas principales
1-2	Análisis y diseño	Requisitos, arquitectura, E-R, wireframes
3-4	Infraestructura	Setup Android (Hilt, Room, Compose), setup Ktor
5	Autenticación	BCrypt, JWT, Google Sign-In, OTP email
6-7	Tareas y proyectos	CRUD completo, tablero Kanban, WebSocket
8-10	Editor colaborativo	Motor CRDT, editor de bloques, WebSocket CRDT
11-12	Workspaces y chat	Workspaces, invitaciones, chat integrado
13-14	Pulido y pruebas	Animaciones M3, modo oscuro, CI/CD, tests
15	Documentación	Redacción del documento PIM

Dedicación total estimada: 350 horas

9. Conclusiones

9.1 Objetivos alcanzados

Mirando el resultado final, lo que más me sorprende es que el proyecto acabó siendo bastante más completo de lo que tenía en mente al principio. El objetivo mínimo era tener tareas y documentos básicos funcionando. Lo que salió fue un editor de bloques con 13 tipos de bloque, edición colaborativa en tiempo real con cursores de otros usuarios visibles, un agente de IA integrado, chat entre usuarios, workspaces de equipo, calendario, papelera con restauración, importación de Markdown y exportación a PDF nativo. No todo estaba planificado desde el día uno.

Lo que más trabajo dio, y de lo que más satisfecho estoy, es el motor CRDT. Tener dos instancias de la app editando el mismo bloque al mismo tiempo sin que los cambios se pierdan o dupliquen no es

trivial. Hay una casuística grande de operaciones concurrentes que hay que resolver de forma determinista, y entender eso a fondo me obligó a leer bastante más allá del temario del ciclo.

En cuanto a los objetivos formales del proyecto, todos están cubiertos: la persistencia dual (PostgreSQL en servidor + Room en cliente), la interfaz declarativa con Compose y Material 3, los WebSockets para sincronización, el backend asíncrono con Ktor, el Google Sign-In con Credential Manager, el despliegue en Render.com con CI/CD y el análisis de sostenibilidad tecnológica. Hay cosas que no llegaron a tiempo y han quedado en §9.3 como mejoras futuras — principalmente las notificaciones push con FCM y el modo offline completo para escritura —, pero son extensiones de funcionalidad, no huecos en lo que se planteó entregar.

9.2 Dificultades encontradas

1. **Motor CRDT:** El caso de dos usuarios editando la misma posición del mismo bloque fue el más complejo. Solución: usar el ID de operación como desempate determinista.
2. **URL de Render.com:** El formato `postgresql://` debe convertirse a `jdbc:postgresql://` con hostname externo, ya que el plan gratuito no tiene red privada.
3. **Inicialización lazy de la DB:** Durante el build Docker la base de datos no es accesible, por lo que la inicialización se difirió al evento `ApplicationStarted`.
4. **Estado WebSocket:** Gestionar reconexiones automáticas y presencia de usuarios requirió un `StateFlow` dedicado para el estado de conexión.

9.3 Mejoras futuras

La exportación a PDF, contemplada inicialmente como mejora futura, ya está implementada mediante `android.graphics.pdf.PdfDocument` (§5.3).

- **FCM (Firebase Cloud Messaging):** Notificaciones push reales con la app en segundo plano. Actualmente las notificaciones son locales y dependen de la app en primer plano.
- **Modo offline completo:** Sincronizar operaciones pendientes (`PendingOperationEntity` ya creada en Room) al recuperar conexión. La lectura offline ya funciona; la escritura requiere conexión activa.
- **Cliente web:** Compose Multiplatform o React para acceso desde navegador, manteniendo el backend Ktor existente.
- **Expiración de JWT:** Implementar refresh tokens con ventana de expiración para mejorar la seguridad.
- **Cifrado E2E:** Cifrado de extremo a extremo para chat y documentos privados, usando la infraestructura JWT ya existente como base.

9.4 Análisis de sostenibilidad

Tecnologías utilizadas y sus alternativas en software libre

Una de las primeras decisiones de FlowBoard fue optar por un stack completamente basado en software libre o de código abierto. Trabajando en GFT Technologies, donde los clientes son principalmente entidades financieras con requisitos estrictos de auditoría y reducción de costes de licencia, he podido comprobar de primera mano que la elección tecnológica no es solo técnica: tiene un impacto económico real.

El análisis de las tecnologías principales del proyecto frente a sus alternativas libres es el siguiente:

Tecnología elegida	Licencia	Alternativa libre	Motivo de la elección
Kotlin	Apache 2.0 (JetBrains/Google)	Dart/Flutter, Java	Es el lenguaje oficial de Android desde 2019; mejor interoperabilidad con Jetpack
Ktor	Apache 2.0 (JetBrains)	Spring Boot, Node.js/Express	Stack Kotlin puro, sin overhead de Spring; menor consumo de memoria en Render.com
PostgreSQL	BSD (completamente libre)	MySQL/MariaDB, SQLite	Mejor soporte de tipos JSON, extensibilidad y conformidad con SQL estándar
Room / SQLite	Apache 2.0 (Google/Android)	SQLDelight (Cash App)	Integración nativa con Jetpack Compose y LiveData/StateFlow
JWT (RFC 7519)	Estándar abierto	OAuth 2.0 completo	Simplicidad para un backend de tamaño medio; OAuth añadiría complejidad sin beneficio claro

En términos económicos, el coste de licencias de este stack es cero. El TCO (Total Cost of Ownership) re-

cae en horas de desarrollo y en infraestructura: Render.com plan gratuito para el MVP, con posibilidad de escalar a planes de pago si hubiera crecimiento real de usuarios. Este modelo es especialmente relevante para una startup o equipo pequeño: las grandes tecnológicas permiten arrancar sin inversión inicial, aunque traen el riesgo de vendor lock-in que comentaré a continuación.

Reflexión sobre el uso de grandes tecnológicas

No todo el stack es completamente neutral. Android como plataforma depende de Google, Kotlin es desarrollado por JetBrains con fuerte influencia de Google, y Cloudinary es un SaaS propietario para el almacenamiento de imágenes. La pregunta honesta es: ¿por qué elegirlos si existen alternativas más independientes?

La respuesta tiene dos capas. La primera es pragmática: en el tiempo disponible para un proyecto de FP, usar el ecosistema oficial de Android con su documentación, sus Codelabs y su comunidad activa reduce el tiempo de resolución de problemas a horas en lugar de días. La segunda la entiendo mejor desde mi formación en economía: hay un efecto red enorme detrás de estas tecnologías. Millones de desarrolladores generan Stack Overflow, repositorios de ejemplo, plugins y actualizaciones que reducen el riesgo técnico del proyecto. Un proyecto pequeño que elige una alternativa más libre pero con menor comunidad está aceptando un coste de soporte potencialmente mayor.

El riesgo real que sí hay que reconocer es el de **dependencia tecnológica**. Si Google deprecara Android SDK mañana, habría que reescribir el cliente desde cero. Si Render.com cerrara su plan gratuito, el backend dejaría de funcionar. En GFT veo este debate constantemente con los clientes bancarios: migrar de soluciones propietarias a cloud tiene costes de salida enormes. Para FlowBoard la solución a largo plazo sería dockerizar completamente el backend (ya está hecho) y sustituir Cloudinary por MinIO o Nextcloud, ambos open source y auto-alojables.

Cifrado, privacidad y protección de datos

FlowBoard maneja datos personales de sus usuarios (email, nombre, historial de actividad), lo que lo sitúa dentro del ámbito del RGPD. El diseño de seguridad utiliza dos tipos de criptografía bien diferenciados:

Cifrado simétrico — HMAC-SHA256 (JWT): El servidor firma cada token JWT con una clave secreta compartida (JWT_SECRET). Es simétrico porque tanto el emisor como el verificador usan la misma clave. La principal ventaja es la velocidad: verificar un JWT es una operación de microsegundos. El riesgo es que si la clave se filtra, todos los tokens emitidos quedan comprometidos. Por eso se almacena como variable de entorno en Render.com, nunca en el código fuente.

Cifrado asimétrico — TLS/HTTPS: Toda la comunicación cliente-servidor usa HTTPS con certificado gestionado automáticamente por Render.com vía Let's Encrypt (CA abierta y gratuita). El handshake TLS utiliza criptografía asimétrica (RSA o ECDH) para intercambiar una clave de sesión simétrica (AES), combinando la seguridad de la asimetría con la velocidad de la simetría.

Hashing unidireccional — BCrypt: Las contraseñas nunca se almacenan en texto plano ni cifradas: se almacena su hash BCrypt. BCrypt incorpora un *salt* aleatorio y un factor de coste configurable que hace que cada cálculo de hash tarde decenas de milisegundos. Esto convierte el cracking por fuerza bruta en algo computacionalmente inviable incluso si la base de datos quedara expuesta.

Desde mi perspectiva con formación en economía: la privacidad tiene un valor económico creciente. Las multas del RGPD pueden alcanzar el 4 % de la facturación global anual. En GFT, los proyectos con datos financieros tienen revisiones de seguridad obligatorias antes del despliegue. FlowBoard, aunque es un proyecto académico, está diseñado con estas mismas premisas.

Plan de formación para usuarios de FlowBoard

Si FlowBoard se desplegara en una organización real (por ejemplo, un equipo de desarrollo o una pequeña consultoría), el plan de incorporación de nuevos usuarios sería el siguiente:

Fase 1 — Presentación (1 h): Sesión grupal de demostración en vivo. Se muestra el flujo completo: registro, creación de workspace, invitar a compañeros, crear una tarea y un documento colaborativo. Sin tecnicismos: el objetivo es que el usuario vea que la app es tan intuitiva como WhatsApp + Notion.

Fase 2 — Taller práctico (2 h): Cada participante instala el APK, crea su cuenta y completa tres mini-retos: (a) crear una tarea con prioridad ALTA y asignarla a un compañero, (b) crear un documento con al menos tres tipos de bloque distintos y compartirlo, (c) unirse al workspace del equipo con el código de invitación.

Fase 3 — Documentación de apoyo (permanente): Acceso al manual básico de usuario del §7.4 de este documento. Canal de soporte mediante el chat integrado de la propia aplicación. FAQs publicadas en el README del repositorio GitHub.

Este plan sigue el modelo de formación que he observado en GFT para herramientas internas: una demo ejecutiva, un taller práctico con objetivos medibles y documentación accesible. La clave es que el usuario tenga un resultado tangible al final del taller, no solo conocimiento teórico.

9.5 Competencias profesionales y reflexión de empleabilidad

Contexto profesional durante el desarrollo del proyecto

Este proyecto intermodular se ha desarrollado en paralelo a mi incorporación laboral en **GFT Technologies**, una consultora especializada en transformación digital para el sector financiero. Esa doble realidad —estudiar DAM mientras trabajo en un entorno de desarrollo profesional— ha marcado profundamente cómo he abordado FlowBoard, tanto en decisiones técnicas como en la organización del trabajo.

Tener acceso a equipos de desarrollo reales me ha permitido contrastar directamente lo que aprendía en el ciclo con las prácticas de la industria: cómo se gestionan las ramas en Git en proyectos grandes, qué significa realmente una arquitectura Clean, por qué el testing no es opcional. También me ha dado perspectiva sobre lo que valoran las empresas de un perfil junior: que el código sea legible, que sepa comunicar avances, y que no bloquee al equipo cuando algo no sale.

Gestión del tiempo y planificación

Compaginar trabajo, estudios y un proyecto de esta envergadura ha requerido una planificación rígida. Dividí el desarrollo en sprints de dos semanas con un límite claro de horas semanales dedicadas al proyecto. Usé GitHub Issues como backlog y configuré recordatorios para las retrospectivas, un hábito que tomé directamente de los dailies de mi equipo en GFT.

Hubo semanas donde la carga de trabajo en la empresa fue mayor y tuve que renegociar el alcance del sprint: mover funcionalidades al siguiente sprint en lugar de entregar algo a medias. Aprendí que entregar menos y bien es preferible a entregar más y mal. Eso es algo que en economía se llama coste de oportunidad: el tiempo que gastas repasando código mal hecho es tiempo que no inviertes en avanzar.

Gestión emocional ante dificultades técnicas

El problema más frustrante del proyecto fue el Google Sign-In en dispositivo físico. Funcionaba perfecto en el emulador durante semanas y fallaba en silencio en el móvil real. El error (`GetCredentialUnknownException`, código 10) tardó días en ser diagnosticado correctamente: el SHA-1 del keystore generado en CI no estaba registrado en Google Cloud Console.

En ese momento la tentación fue abandonar la funcionalidad y marcarla como “no disponible en builds de CI”. No lo hice. Busqué de forma sistemática, encontré la causa raíz y lo resolví correctamente. Esa decisión —no parchear sino resolver— es algo que también he aprendido en el entorno

laboral: los atajos técnicos generan deuda que alguien pagará después, y en un equipo pequeño ese alguien eres tú mismo.

Comunicación y trabajo en equipo simulado

FlowBoard es un proyecto individual, pero desde el primer sprint lo traté como si tuviera un cliente real. Redacté los requisitos como historias de usuario, definí criterios de aceptación concretos y mantuve el documento de memoria actualizado a medida que el código evolucionaba. Ese ejercicio de documentar mientras desarrollas, no al final, es algo que en GFT se valora enormemente porque reduce el tiempo de onboarding de nuevos miembros.

Mi formación previa en el primer año de Economía en la Universidad de Valencia también ha influido en cómo presento este proyecto. Saber construir un argumento estructurado, entender el coste real de las decisiones técnicas en términos económicos, y comunicar ideas a audiencias no técnicas son capacidades que el grado en Economía desarrolla bien y que el DAM complementa con las herramientas para ejecutarlas.

Competencias identificadas como áreas de mejora

Siendo honesto, hay competencias donde sé que necesito crecer. El testing automatizado no es algo que hiciera de forma natural: los tests de este proyecto los añadí al final, cuando deberían haber guiado el desarrollo desde el principio (TDD). La gestión de la incertidumbre también es un área de desarrollo: cuando un problema técnico no tiene solución inmediata visible, mi tendencia inicial es acumular horas de búsqueda en solitario en lugar de pedir ayuda antes. Eso es algo que estoy trabajando en el entorno laboral.

10. Anexos

10.1 Manual técnico

Requisitos del entorno de desarrollo

Backend (Ktor):

Herramienta	Versión mínima
JDK	17

Herramienta	Versión mínima
Kotlin	1.9.x
Gradle	8.x
Docker	24.x (para despliegue en Render.com)

Cliente Android:

Herramienta	Versión mínima
Android Studio	Iguana (2023.2.1)
JDK	17
Android SDK	API 26+
Gradle	8.x

Configuración del entorno local

1. Clonar el repositorio:

```
git clone https://github.com/paulopnun/flowboard.git
cd flowboard
```

2. Configurar variables de entorno del backend (archivo backend/.env):

```
DATABASE_URL=jdbc:postgresql://localhost:5432/flowboard
DATABASE_USER=flowboard
DATABASE_PASSWORD=flowboard
JWT_SECRET=your-32-char-minimum-secret-key-here
JWT_ISSUER=flowboard-api
JWT_AUDIENCE=flowboard-app
RESEND_API_KEY=re_your_resend_api_key
```

3. Arrancar el backend localmente:

```
cd backend
./gradlew run
# Server starts at http://localhost:8080
```

4. Apuntar el cliente Android al backend local (NetworkModule.kt):


```
// 10.0.2.2 is the Android emulator alias for localhost
const val BASE_URL = "http://10.0.2.2:8080/api/v1/"
```

5. Ejecutar el cliente Android: Abrir la carpeta android/ en Android Studio → seleccionar emulador o dispositivo → **Run** (Shift+F10).

Esquema de autenticación

Todas las rutas protegidas (columna Auth = “Sí”) requieren el header HTTP:

Authorization: Bearer <JWT_TOKEN>

El token se obtiene en /auth/login o /auth/register.

Referencia completa de la API REST

Base URL: <https://flowboard-api-phrk.onrender.com/api/v1>

Recurso	Método	Endpoint	Descripción
Auth	POST	/auth/register	Registrar usuario
Auth	POST	/auth/login	Login email/contraseña
Auth	POST	/auth/google	Login con Google
Auth	POST	/auth/logout	Cerrar sesión
Auth	POST	/auth/forgot-password	Solicitar OTP
Auth	POST	/auth/reset-password	Confirmar OTP + nueva contraseña
Tasks	GET	/tasks	Listar tareas del usuario
Tasks	GET	/tasks/{id}	Obtener tarea
Tasks	POST	/tasks	Crear tarea
Tasks	PUT	/tasks/{id}	Actualizar tarea
Tasks	DELETE	/tasks/{id}	Eliminar tarea

Recurso	Método	Endpoint	Descripción
Tasks	PATCH	/tasks/{id}/toggle	Alternar completado
Tasks	GET	/tasks/events	Eventos entre fechas
Projects	GET	/projects	Listar proyectos
Projects	POST	/projects	Crear proyecto
Projects	GET	/projects/{id}	Obtener proyecto
Projects	PUT	/projects/{id}	Actualizar proyecto
Projects	DELETE	/projects/{id}	Eliminar proyecto
Documents	GET	/documents	Listar documentos
Documents	POST	/documents	Crear documento
Documents	GET	/documents/{id}	Obtener documento
Documents	PUT	/documents/{id}	Actualizar documento
Documents	DELETE	/documents/{id}	Eliminar documento
Workspaces	GET	/workspaces	Listar workspaces
Workspaces	POST	/workspaces	Crear workspace
Workspaces	POST	/workspaces/join	Unirse con código
Chat	GET	/chat/rooms	Listar salas
Chat	POST	/chat/rooms	Crear sala
Chat	GET	/chat/rooms/{id}/messages	Mensajes de sala
Notifications	GET	/notifications	Listar notificaciones
Notifications	PATCH	/notifications/{id}/read	Marcar como leída
Permissions	POST	/permissions/document	Compartir documento
WebSocket	WS	/ws/{boardId}	Eventos de tareas en tiempo real
WebSocket	WS	/document-ws/{docId}	CRDT de documentos

10.2 Repositorio y recursos

- **Repositorio GitHub:** <https://github.com/paulopnun/flowboard>

- **Backend producción:** <https://flowboard-api-phrk.onrender.com>
- **APK:** Sección *Releases* del repositorio (generado automáticamente por GitHub Actions en cada push a master)

10.3 Referencias y recursos utilizados

Recursos de aprendizaje consultados

- **Android Developers Codelabs** — <https://developer.android.com/codelabs>
Tutoriales oficiales de Jetpack Compose, Hilt, Room y DataStore utilizados durante el desarrollo.
- **Kotlin Playground** — <https://play.kotlinlang.org>
Verificación rápida de fragmentos de lógica Kotlin, especialmente para el motor CRDT.
- **Stack Overflow** — <https://stackoverflow.com>
Resolución de problemas específicos: configuración de HikariCP con Render.com, manejo de WebSockets en Ktor, y serialización de tipos genéricos con Exposed ORM.

Repositorios de referencia

- **JetBrains/Exposed** — <https://github.com/JetBrains/Exposed>
ORM Kotlin utilizado para el acceso a PostgreSQL en el servidor Ktor.
- **square/okhttp** — <https://github.com/square/okhttp>
Cliente WebSocket subyacente al engine OkHttp de Ktor en Android.
- **coil-kt/coil** — <https://github.com/coil-kt/coil>
Librería de carga de imágenes asíncrona para Compose, usada en portadas de documentos y avatares.

Estándares y especificaciones

- **Material Design 3 Guidelines** — <https://m3.material.io>
Referencia oficial para el sistema de diseño, tokens de color, tipografía y radios de curvatura.
- **RFC 7519 — JSON Web Token** — <https://datatracker.ietf.org/doc/html/rfc7519>
Especificación del estándar JWT implementado en el backend con Auth0 java-jwt.
- **RFC 6455 — The WebSocket Protocol** — <https://datatracker.ietf.org/doc/html/rfc6455>
Especificación del protocolo WebSocket para la edición colaborativa en tiempo real.

10.4 Cumplimiento de Resultados de Aprendizaje (PIM)

Módulo	RA	Funcionalidad en FlowBoard / BIT
Acceso a Datos	RA02	PostgreSQL + Exposed ORM + HikariCP + Room SQLite + DataStore + trigger <code>set_updated_at</code> (procedimiento almacenado PL/pgSQL)
Desarrollo de Interfaces	RA01	Jetpack Compose + NavHost 20+ rutas + Material 3 + sidebar colapsable estilo Notion
Desarrollo de Interfaces	RA02	Color.kt (paleta light/dark), Type.kt (Inter, 13 estilos)
Desarrollo de Interfaces	RA03	<code>spring(DampingRatioMediumBouncy)</code> , <code>tween(280ms)</code> , <code>animateColorAsState</code> , <code>animateItem</code> , <code>infiniteTransition</code> , <code>rememberSaveable</code> , <code>ContentScale.Crop</code> , <code>Modifier.weight</code>
Desarrollo de Interfaces	RA05	Exportación de documentos: informe PDF estructurado (<code>DocumentExporter.kt</code>) con fuentes, paginación y filtrado por tipo de bloque; informe Markdown compatible

Módulo	RA	Funcionalidad en FlowBoard / BIT
Desarrollo de Interfaces	RA06	contentDescription en todos los iconos interactivos; semántica Compose (semantics { }) para accesibilidad; documentación técnica en Markdown (este documento)
Prog. Multimedia y Móviles	RA03	Motor CRDT + WebSocket en tiempo real + Coil (imágenes/vídeo/audio) + notificaciones del sistema con deep links
Prog. Multimedia y Móviles	RA04	Proyecto BIT — análisis de motores Unity 6, arquitectura de juego 2D top-down, componentes (Rigidbody2D, Collider2D, Animator, Camera), FSM de enemigos, ScriptableObjects. Repositorio: https://github.com/PauLopNun/BIT
Prog. Multimedia y Móviles	RA05	Proyecto BIT — juego 2D completo con sistema de oleadas, 3 personajes jugables, 5 tipos de enemigos con IA (FSM), audio BGM + 5 SFX, ranking JSON local, despliegue en dispositivo móvil (New Input System). Repositorio: https://github.com/PauLopNun/BIT
Prog. Procesos y Servicios	RA04	Ktor asíncrono + coroutines + 40+ endpoints REST

Módulo	RA	Funcionalidad en FlowBoard / BIT
Prog. Procesos y Servicios	RA05	JWT HMAC-256 + BCrypt + roles viewer/editor/owner
Empleabilidad II	RA02	Google Sign-In con Credential Manager
Sostenibilidad	RA06	HikariCP pooling + lazy init DB + WorkManager
Nube Pública	RA04	Render.com (backend + PostgreSQL) + CI/CD GitHub Actions

Nota sobre RA04 y RA05 de Programación Multimedia y Dispositivos Móviles: estos resultados de aprendizaje, relativos al análisis de motores de juegos y al desarrollo de juegos 2D/3D, se cubren mediante el proyecto **BIT**, desarrollado en paralelo como parte del mismo módulo durante el presente curso. BIT es un juego 2D top-down de acción y supervivencia construido con Unity 6, que implementa generación procedural de mazmorras, inteligencia artificial de enemigos mediante FSM, sistema de oleadas escalable, animaciones por sprites, física con RigidBody2D y despliegue en dispositivo móvil. El repositorio completo está disponible en <https://github.com/PauLopNun/BIT>. FlowBoard cubre RA03 del mismo módulo con el motor CRDT, los WebSockets y la gestión de multimedia en el editor colaborativo.

11. Bibliografía

11.1 Referencias bibliográficas en formato APA

- Android Developers. (2025). *Jetpack Compose documentation*. Google. <https://developer.android.com/jetpack/compose>
- Android Developers. (2025). *Room Persistence Library*. Google. <https://developer.android.com/training/data-storage/room>
- Android Developers. (2025). *Hilt dependency injection*. Google. <https://developer.android.com/training/dependency-injection/hilt-android>
- Android Developers. (2025). *Material Design 3 for Android*. Google. <https://m3.material.io/develop/android>

- Android Developers. (2025). *Credential Manager API*. Google. <https://developer.android.com/identity/sign-in/credential-manager>
- JetBrains. (2025). *Ktor documentation*. JetBrains. <https://ktor.io/docs>
- JetBrains. (2025). *Exposed ORM documentation*. JetBrains. <https://github.com/JetBrains/Exposed/wiki>
- JetBrains. (2025). *Kotlin Coroutines Guide*. JetBrains. <https://kotlinlang.org/docs/coroutines-guide.html>
- JetBrains. (2025). *kotlinx.serialization*. JetBrains. <https://kotlinlang.org/docs/serialization.html>
- Shapiro, M., Preguiça, N., Baquero, C., & Zawirski, M. (2011). *Conflict-free Replicated Data Types*. INRIA Research Report RR-7687. <https://hal.inria.fr/inria-00609399>

11.2 Recursos en línea (con URLs y fecha de consulta)

- Auth0. (2025). *Introduction to JSON Web Tokens*. <https://jwt.io/introduction> [Consultado el 10 de enero de 2026]
- Render. (2025). *Deploying applications on Render*. <https://render.com/docs> [Consultado el 15 de noviembre de 2025]
- Google. (2025). *Sign in with Google for Android*. <https://developers.google.com/identity/sign-in/android> [Consultado el 5 de diciembre de 2025]
- Resend. (2025). *Resend Email API documentation*. <https://resend.com/docs> [Consultado el 20 de noviembre de 2025]
- Internet Engineering Task Force. (2015). *RFC 7519 — JSON Web Token (JWT)*. <https://datatracker.ietf.org/doc/html/rfc7519> [Consultado el 12 de enero de 2026]
- Internet Engineering Task Force. (2011). *RFC 6455 — The WebSocket Protocol*. <https://datatracker.ietf.org/doc/html/rfc6455> [Consultado el 8 de febrero de 2026]
- Google. (2025). *Material Design 3 — Design tokens*. <https://m3.material.io/foundations/design-tokens/overview> [Consultado el 3 de octubre de 2025]
- Android Developers. (2025). *DataStore documentation*. <https://developer.android.com/topic/libraries/architecture/datastore> [Consultado el 18 de octubre de 2025]
- Android Developers. (2025). *WorkManager documentation*. <https://developer.android.com/topic/libraries/architecture/workmanager> [Consultado el 22 de octubre de 2025]

11.3 Documentación de librerías / frameworks utilizadas

- Zaxxer. (2025). *HikariCP — High-performance JDBC connection pool* [Software]. <https://github.com/brettwooldridge/HikariCP> [Consultado el 5 de noviembre de 2025]

- Auth0. (2025). *java-jwt* [Software]. <https://github.com/auth0/java-jwt> [Consultado el 10 de enero de 2026]
- mindrot. (2024). *jBCrypt* [Software]. <https://www.mindrot.org/projects/jBCrypt/> [Consultado el 8 de noviembre de 2025]
- Coil Contributors. (2025). *Coil — Image loading for Android and Compose* [Software]. <https://coil-kt.github.io/coil/> [Consultado el 15 de octubre de 2025]
- Square. (2025). *OkHttp* [Software]. <https://square.github.io/okhttp/> [Consultado el 12 de octubre de 2025]
- Google. (2025). *Hilt — Dependency injection with Hilt* [Software]. <https://dagger.dev/hilt/> [Consultado el 20 de septiembre de 2025]
- JetBrains. (2025). *kotlinx-datetime* [Software]. <https://github.com/Kotlin/kotlinx-datetime> [Consultado el 18 de noviembre de 2025]

Documento elaborado conforme a la rúbrica del Proyecto Intermodular (PIM) — 2.º DAM — IES La Sénia, curso 2025–2026. Todo el contenido técnico está basado en el código real del repositorio <https://github.com/paulopnun/flowboard>.